

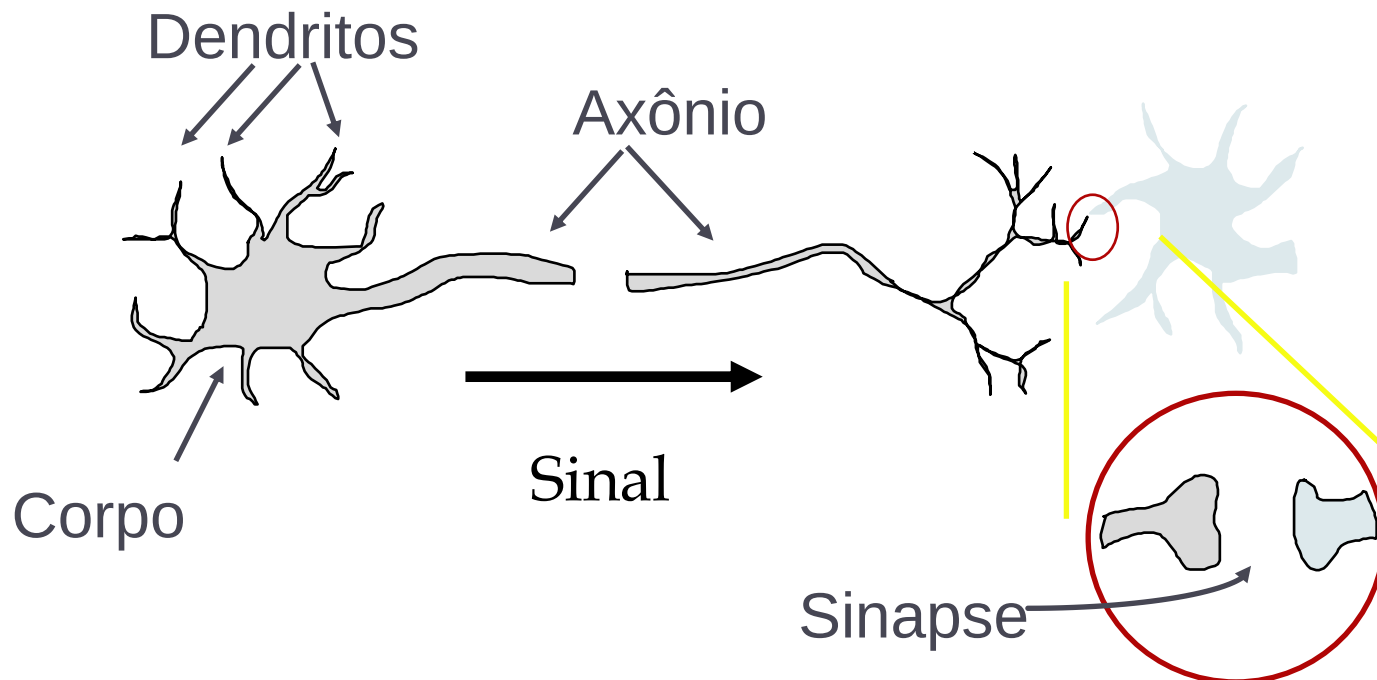


Redes Neurais

Prof. Bruno Nogueira

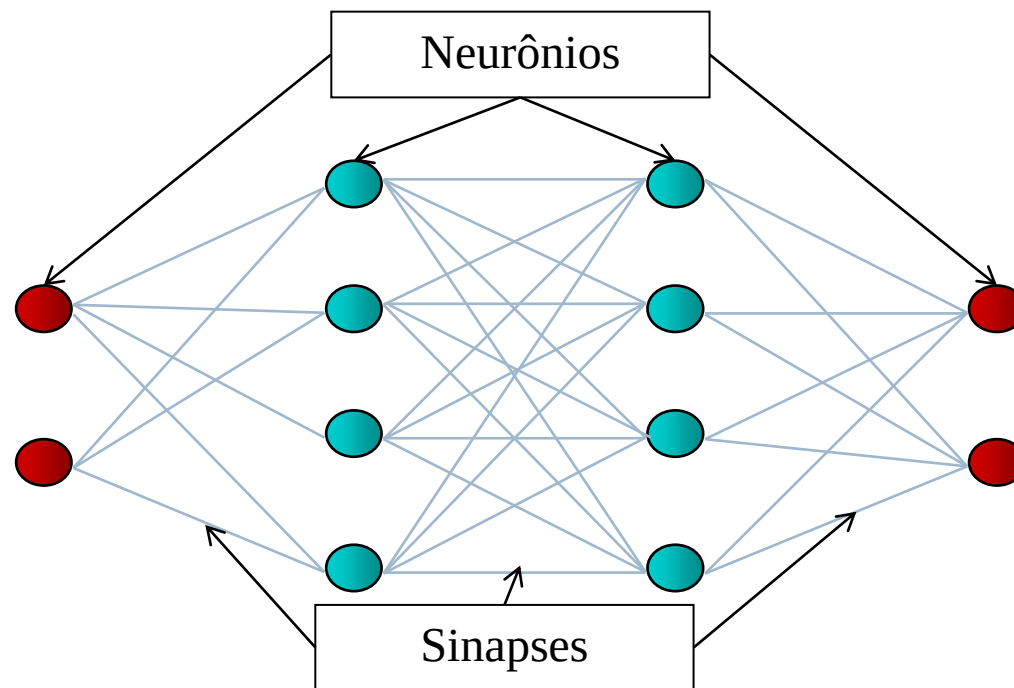
O que são Redes Neurais?

- ▶ Modelos inspirados no cérebro humano
 - ▶ Compostos por unidades de processamento (**neurônios**)
 - ▶ Interligadas por um grande número de conexões (**sinapses**)



O que são Redes Neurais?

- ▶ Modelos inspirados no cérebro humano
 - ▶ Compostos por unidades de processamento (**neurônios**)
 - ▶ Interligadas por um grande número de conexões (**sinapses**)

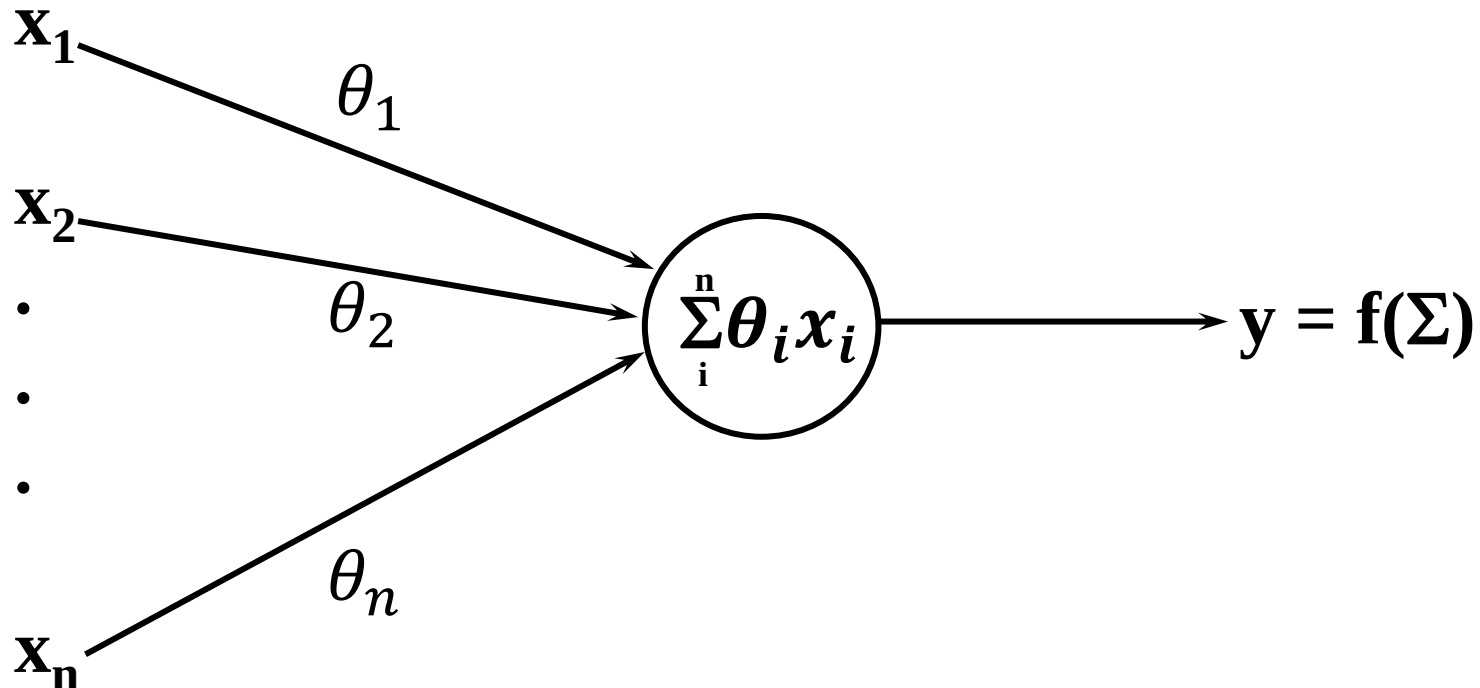


Redes Neurais Artificiais (RNA)

- ▶ Tentativas de sistemas de aprendizado biologicamente realistas, baseadas em modelos abstratos sobre o cérebro
- ▶ Na prática, funcionam de forma inteiramente diferente
 - ▶ Neurônios são $\approx$ 1 milhão de vezes mais lentos que as portas lógicas de silício
 - ▶ Lentidão compensada por grande número de neurônios massivamente conectados
 - ▶ Para certas operações, o cérebro humano é muito mais rápido que computadores convencionais
 - ▶ Visão, audição, controle, previsão

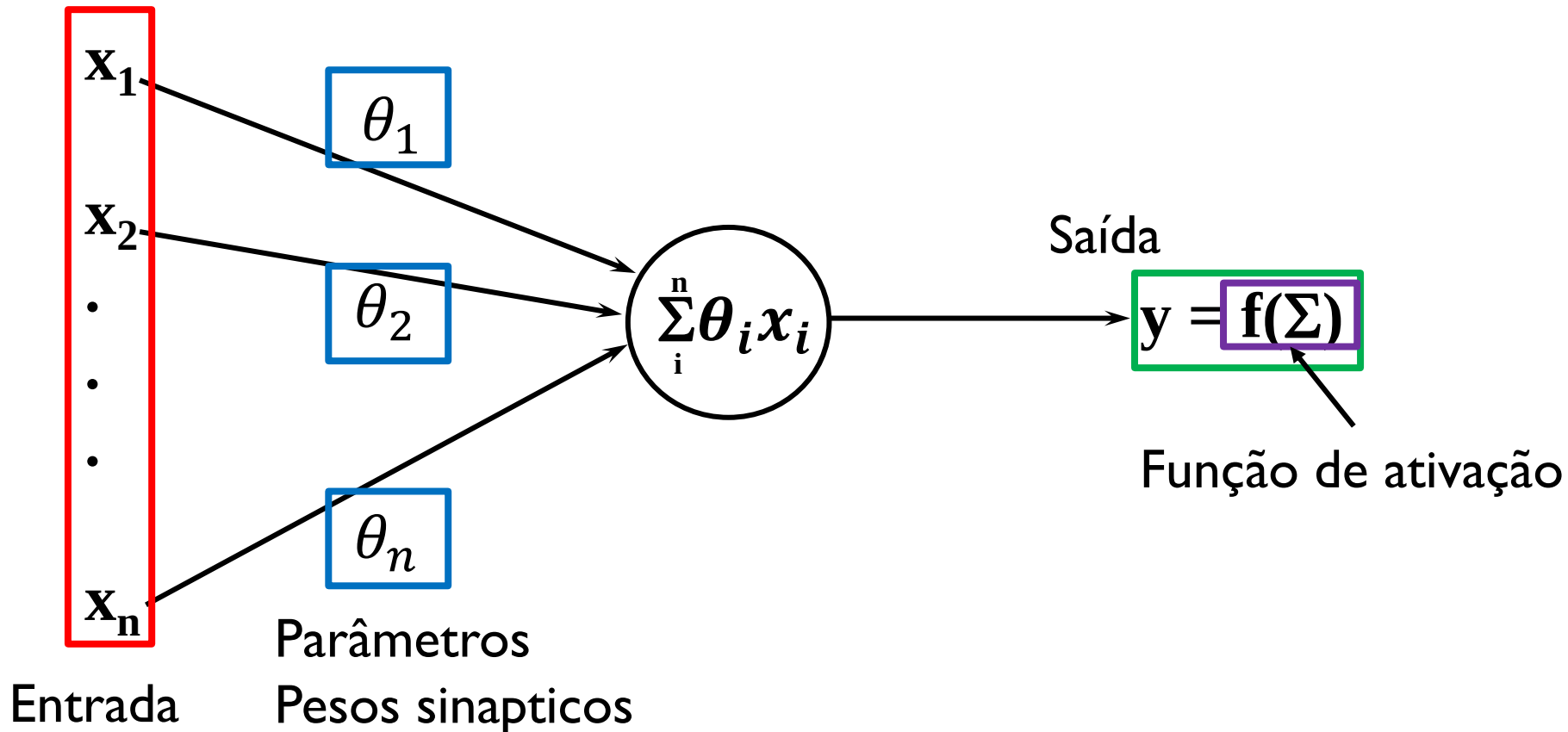
Neurônio artificial

- Também conhecido como unidade de processamento

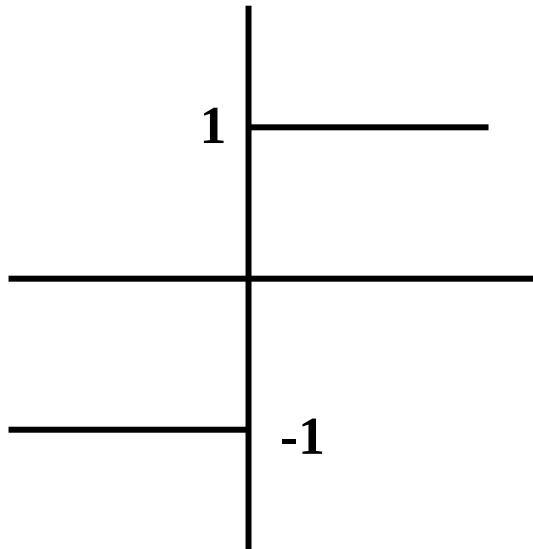


Neurônio artificial

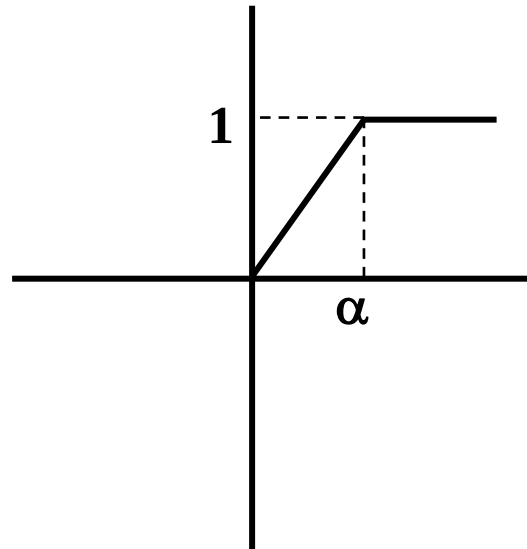
- ▶ Também conhecido como unidade de processamento



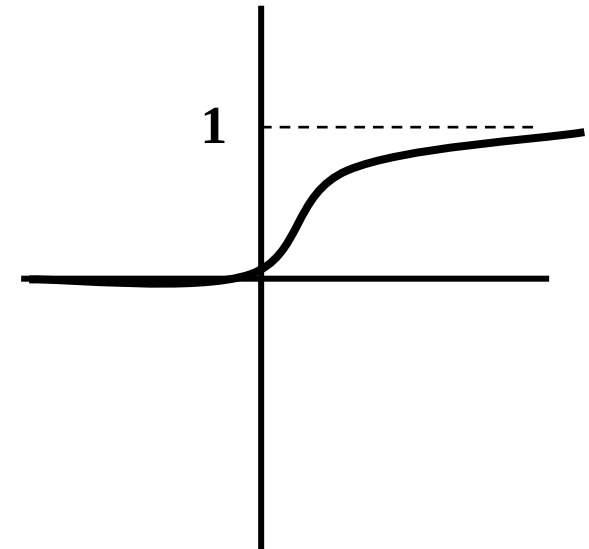
Funções de ativação



Hard Limiter
Degrau

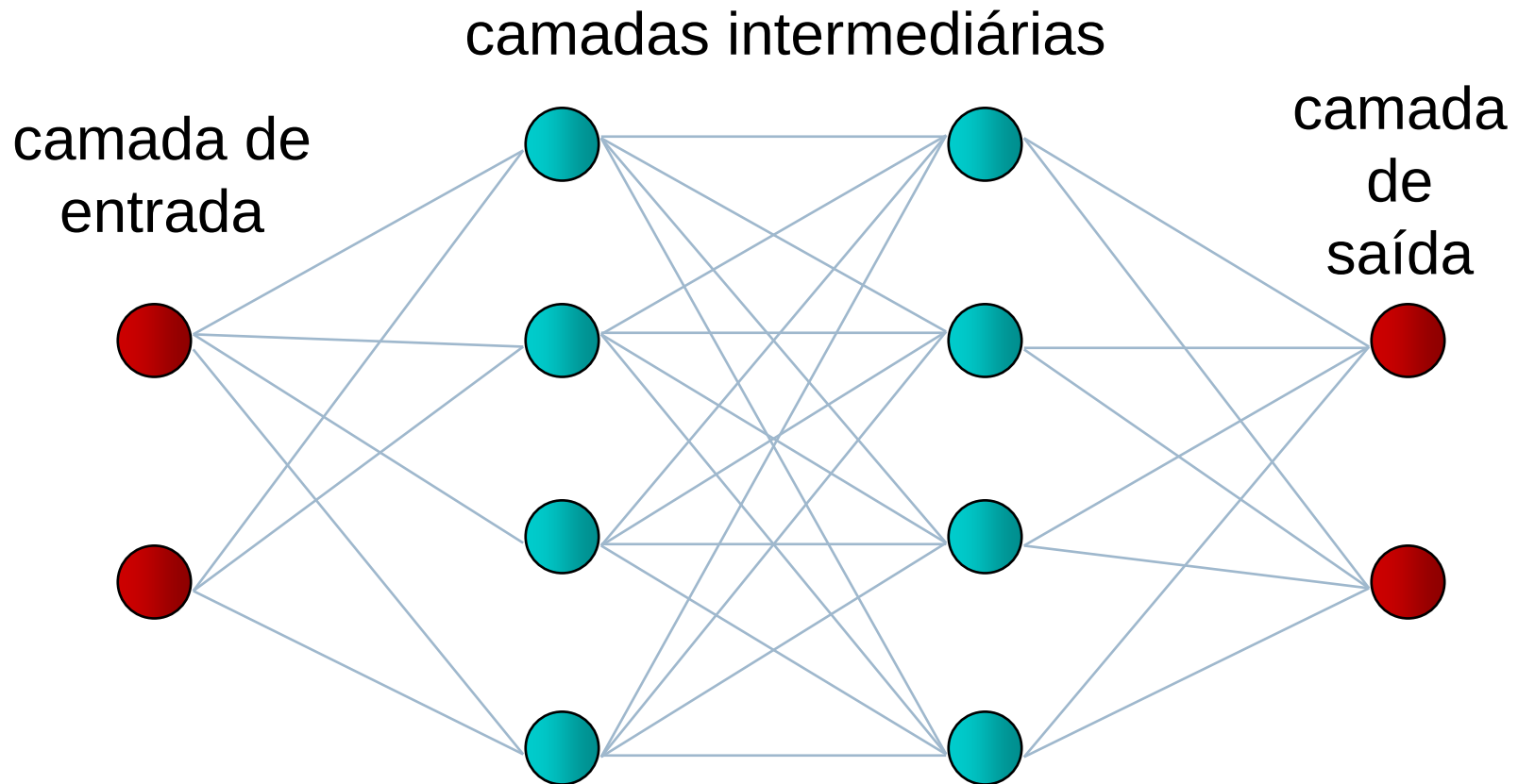


Threshold Logic



Sigmoid

Arquitetura das RNA's

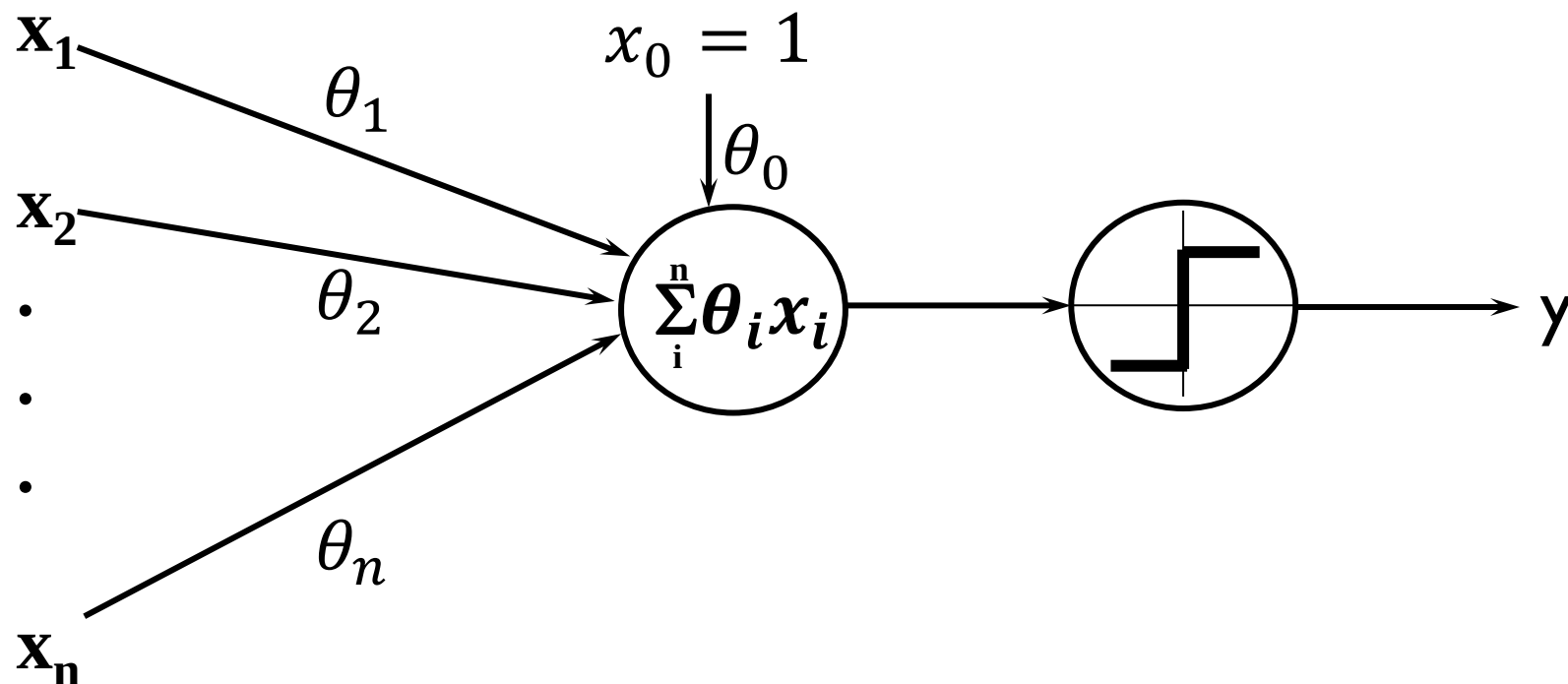


Aprendizado em RNA's

- ▶ Assim como nos algoritmos de Regressão Linear e Regressão Logística, o aprendizado nas RNA's consiste em aprender o conjunto de parâmetros θ que melhor se adeque ao problema
- ▶ Algoritmo de aprendizado depende do modelo de neurônio utilizado
 - ▶ RNA = Arquitetura + Aprendizado
 - ▶ Arquitetura é definida pela disposição dos neurônios na rede
 - ▶ Número de “camadas”
 - ▶ Número de neurônios em cada camada

Perceptron

- Um dos primeiros modelos de neurônio desenvolvidos



saída= 1, se $\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n > 0$
 saída= -1 (ou 0), caso contrário

θ_0 é o *bias*

Perceptron

► Algoritmo de aprendizado

1. Iniciam-se os pesos sinapticos com valores aleatórios pequenos ou iguais a zero
2. Aplica-se um padrão (exemplo) $x^{(i)}$ e gera-se a saída $h(x^{(i)})$
3. Calcula-se o erro na saída: $E^{(i)} = y^{(i)} - h(x^{(i)})$
4. Se $E^{(i)} = 0$, volta-se ao passo 2, aplicando o próximo exemplo; se $E^{(i)} \neq 0$, atualiza-se os pesos

$$\theta_j = \theta_j + \alpha * E^{(i)} * x_j^{(i)}$$

5. Volta ao passo 2.

Perceptron

► Algoritmo de aprendizado

1. Iniciam-se os pesos sinapticos com valores aleatórios pequenos ou iguais a zero
2. Aplica-se um padrão (exemplo) $x^{(i)}$ e gera-se a saída $h(x^{(i)})$
3. Calcula-se o erro na saída: $E^{(i)} = y^{(i)} - h(x^{(i)})$
4. Se $E^{(i)} = 0$, volta-se ao passo 2, aplicando o próximo exemplo; se $E^{(i)} \neq 0$, atualiza-se os pesos

$$\theta_j = \theta_j + \boxed{\alpha} * E^{(i)} * x_j^{(i)}$$

Taxa de aprendizado

5. Volta ao passo 2.

Perceptron - Exemplo

► Simulação do operador lógico AND

Exemplo	x_1	x_2	y
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

► Vamos considerar os pesos iniciais iguais a 0 e uma taxa de aprendizado de 0.5

Perceptron – Exemplo – 1º ciclo

Entrada 1: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(0 \times 1 + 0 \times 0 + 0 \times 0) = h(0) = 0$ $h(x) = y$

Entrada 2: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(0 \times 1 + 0 \times 0 + 0 \times 1) = h(0) = 0$ $h(x) = y$

Entrada 3: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(0 \times 1 + 0 \times 1 + 0 \times 0) = h(0) = 0$ $h(x) = y$

Entrada 4: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(0 \times 1 + 0 \times 1 + 0 \times 1) = h(0) = 0$ $h(x) \neq y$

$$\theta_0 = \theta_0 + \alpha * E^{(4)} * x_0 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$$

$$\theta_1 = \theta_1 + \alpha * E^{(4)} * x_1 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$$

$$\theta_2 = \theta_2 + \alpha * E^{(4)} * x_2 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$$

Perceptron – Exemplo – 2º ciclo

Entrada 1: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(0.5 \times 1 + 0.5 \times 0 + 0.5 \times 0) = h(0.5) = 1 \quad h(x) \neq y$$

$$\theta_0 = \theta_0 + \alpha * E^{(1)} * x_0 = 0.5 + 0.5 \times (0-1) \times 1 = 0$$

$$\theta_1 = \theta_1 + \alpha * E^{(1)} * x_1 = 0.5 + 0.5 \times (0-1) \times 0 = 0.5$$

$$\theta_2 = \theta_2 + \alpha * E^{(1)} * x_2 = 0.5 + 0.5 \times (0-1) \times 0 = 0.5$$

Entrada 2: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(0 \times 1 + 0.5 \times 0 + 0.5 \times 1) = h(0.5) = 1 \quad h(x) \neq y$$

$$\theta_0 = \theta_0 + \alpha * E^{(2)} * x_0 = 0 + 0.5 \times (0-1) \times 1 = -0.5$$

$$\theta_1 = \theta_1 + \alpha * E^{(2)} * x_1 = 0.5 + 0.5 \times (0-1) \times 0 = 0.5$$

$$\theta_2 = \theta_2 + \alpha * E^{(2)} * x_2 = 0.5 + 0.5 \times (0-1) \times 1 = 0$$

Perceptron – Exemplo – 2º ciclo

Entrada 3: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(-0.5 \times 1 + 0.5 \times 1 + 0 \times 0) = h(0) = 0 \quad h(x) = y$

Entrada 4: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(-0.5 \times 1 + 0.5 \times 1 + 0 \times 1) = h(0) = 0 \quad h(x) \neq y$
 $\theta_0 = \theta_0 + \alpha * E^{(4)} * x_0 = -0.5 + 0.5 \times (1 - 0) \times 1 =$
0
 $\theta_1 = \theta_1 + \alpha * E^{(4)} * x_1 = 0.5 + 0.5 \times (1 - 0) \times 1 =$
1
 $\theta_2 = \theta_2 + \alpha * E^{(4)} * x_2 = 0 + 0.5 \times (1 - 0) \times 1 =$
0.5

Perceptron – Exemplo – 3º ciclo

Entrada 1: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(0 \times 1 + 1 \times 0 + 0.5 \times 0) = f(0) = 0 \quad h(x) = y$

Entrada 2: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(0 \times 1 + 1 \times 0 + 0.5 \times 1) = f(0.5) = 1 \quad h(x) \neq y$

$$\theta_0 = \theta_0 + \alpha * E^{(2)} * x_0 = 0 + 0.5 \times (0 - 1) \times 1 = -0.5$$

$$\theta_1 = \theta_1 + \alpha * E^{(2)} * x_1 = 1 + 0.5 \times (0 - 1) \times 0 = 1$$

$$\theta_2 = \theta_2 + \alpha * E^{(2)} * x_2 = 0.5 + 0.5 \times (0 - 1) \times 1 = 0$$

Perceptron – Exemplo – 3º ciclo

Entrada 3: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-0,5 \times 1 + 1 \times 1 + 0 \times 0) = h(0.5) = 1 \quad h(x) \neq y$$

$$\theta_0 = \theta_0 + \alpha * E^{(3)} * x_0 = -0.5 + 0.5 \times (0-1) \times 1 = -1.0$$

$$\theta_1 = \theta_1 + \alpha * E^{(3)} * x_1 = 1 + 0.5 \times (0-1) \times 1 = 0.5$$

$$\theta_2 = \theta_2 + \alpha * E^{(3)} * x_2 = 0 + 0.5 \times (0-1) \times 0 = 0$$

Entrada 4: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-1 \times 1 + 0.5 \times 1 + 0 \times 1) = h(-0.5) = 0 \quad h(x) \neq y$$

$$\theta_0 = \theta_0 + \alpha * E^{(4)} * x_0 = -1 + 0.5 \times (1-0) \times 1 = -0.5$$

$$\theta_1 = \theta_1 + \alpha * E^{(4)} * x_1 = 0.5 + 0.5 \times (1-0) \times 1 = 1$$

$$\theta_2 = \theta_2 + \alpha * E^{(4)} * x_2 = 0 + 0.5 \times (1-0) \times 1 = 0.5$$

Perceptron – Exemplo – 4º ciclo

Entrada 1: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-0.5 \times 1 + 1 \times 0 + 0.5 \times 0) = h(-0.5) = 0 \quad h(x) = y$$

Entrada 2: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-0.5 \times 1 + 1 \times 0 + 0.5 \times 1) = f(0) = 0 \quad h(x) = y$$

Entrada 3: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-0.5 \times 1 + 1 \times 1 + 0.5 \times 0) = f(0.5) = 1 \quad h(x) \neq y$$

$$\theta_0 = \theta_0 + \alpha * E^{(3)} * x_0 = -0.5 + 0.5 \times (0-1) \times 1 = -1$$

$$\theta_1 = \theta_1 + \alpha * E^{(3)} * x_1 = 1 + 0.5 \times (0-1) \times 1 = 0.5$$

$$\theta_2 = \theta_2 + \alpha * E^{(3)} * x_2 = 0.5 + 0.5 \times (0-1) \times 0 = 0.5$$

Perceptron – Exemplo – 4º ciclo

Entrada 4: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-1 \times 1 + 0.5 \times 1 + 0.5 \times 1) = h(0) = 0 \quad h(x) \neq y$$

$$\theta_0 = \theta_0 + \alpha * E^{(4)} * x_0 = -1 + 0.5 \times (1-0) \times 1 = -0.5$$

$$\theta_1 = \theta_1 + \alpha * E^{(4)} * x_1 = 0.5 + 0.5 \times (1-0) \times 1 = 1$$

$$\theta_2 = \theta_2 + \alpha * E^{(4)} * x_2 = 0.5 + 0.5 \times (1-0) \times 1 = 1$$

Perceptron – Exemplo – 5º ciclo

Entrada 1: $s_{out} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-0.5 \times 1 + 1 \times 0 + 1 \times 0) = h(-0.5) = 0$$

$$h(x) = y$$

Entrada 2: $s_{out} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-0.5 \times 1 + 1 \times 0 + 1 \times 1) = f(0.5) = 1$$

$$h(x) \neq y$$

$$\theta_0 = \theta_0 + \alpha * E^{(4)} * x_0 = -0.5 + 0.5 \times (0 - 1) \times 1 = -1$$

$$\theta_1 = \theta_1 + \alpha * E^{(2)} * x_1 = 1 + 0.5 \times (0 - 1) \times 0 = 1$$

$$\theta_2 = \theta_2 + \alpha * E^{(2)} * 2 = 1 + 0.5 \times (0 - 1) \times 1 = 0.5$$

Entrada 3: $s_{out} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-1 \times 1 + 1 \times 1 + 0.5 \times 0) = h(0) = 0$$

$$h(x) = y$$

Entrada 4: $s_{out} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$

$$= h(-1 \times 1 + 1 \times 1 + 0.5 \times 1) = h(0.5) = 1$$

$$h(x) = y$$

Perceptron – Exemplo – 6º ciclo

Entrada 1: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(-1 \times 1 + 1 \times 0 + 0.5 \times 0) = h(-1) = 0$ $h(x) = y$

Entrada 2: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(-1 \times 1 + 1 \times 0 + 0.5 \times 1) = h(-0.5) = 0$ $h(x) = y$

Entrada 3: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(-1 \times 1 + 1 \times 1 + 0.5 \times 0) = h(0) = 0$ $h(x) = y$

Entrada 4: $s_{\text{out}} = h(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2)$
 $= h(-1 \times 1 + 1 \times 1 + 0.5 \times 1) = h(0.5) = 1$ $h(x) = y$

$\theta_0 = -1, \theta_1 = 1, \theta_2 = 0.5$

Exercício

- ▶ Considere a base de dados abaixo

Aluno	Estudou	Fez o trabalho	Passou
Joãozinho	Não	Não	Não
Huguinho	Não	Sim	Não
Zezinho	Sim	Não	Sim
Luizinho	Sim	Sim	Sim

- ▶ Faça duas iterações (dois ciclos) do algoritmo de treinamento do perceptron para este problema, considerando todos os pesos iguais a 0 e a taxa de aprendizado igual a 0,1

Exercício II

- Considerando a base de dados abaixo e os mesmos parâmetros do exercício anterior, faça dois ciclos de treinamento para a base de dados abaixo

	VÍRUS	BACTÉRIA	DOR CABEÇA	CORISA
GRIPE	1	0	1	1
RESFRIADO	0	1	0	1
GRIPE	1	0	1	0
RESFRIADO	0	1	1	1
GRIPE	0	0	1	1
RESFRIADO	0	0	0	1

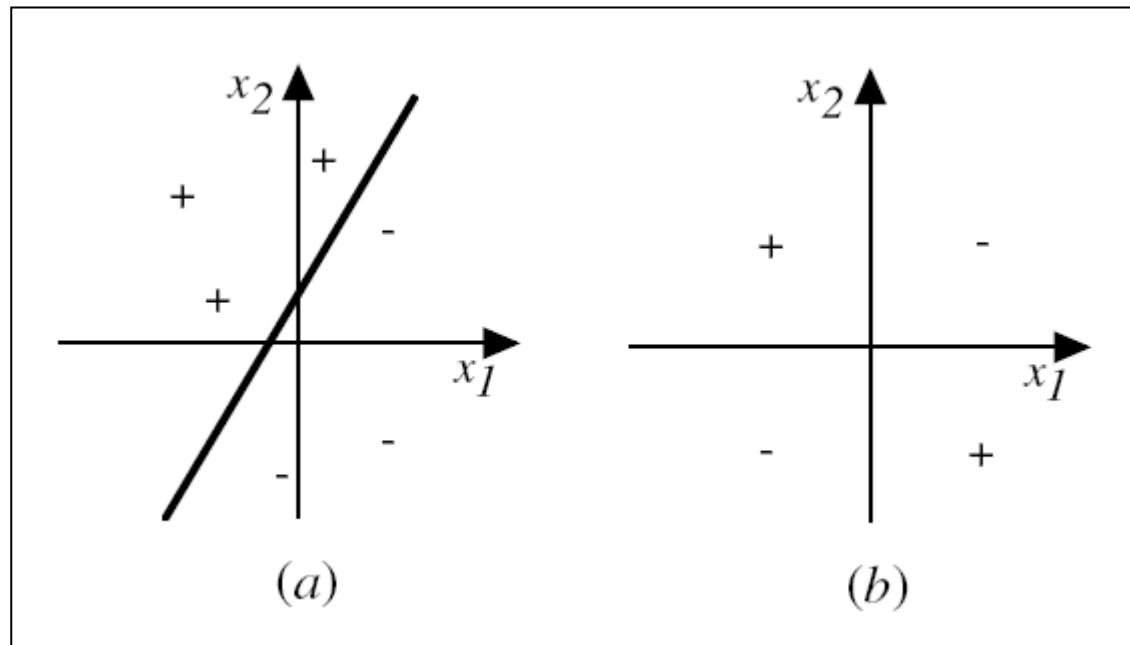
Exercício II (cont)

- ▶ Considerando os parâmetros obtidos no passo anterior, mostre como seria a classificação dos seguintes novos dados nesta rede perceptron

	VÍRUS	BACTÉRIA	DOR CABEÇA	CORISA
?	1	1	1	1
?	1	0	0	0
?	0	1	0	0
?	0	1	1	1
?	1	0	0	1
?	0	0	0	0

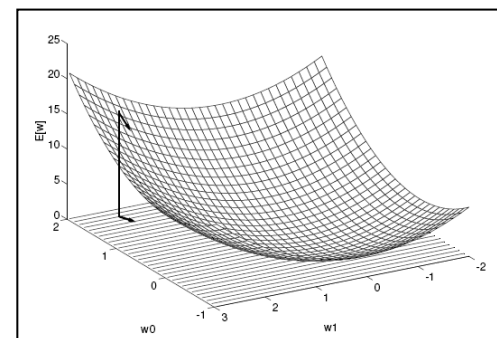
Perceptron - Treinamento

- ▶ A regra de treinamento do perceptron não converge quando o problema não é linearmente separável



Perceptron – Regra Delta

- ▶ Para encontrar a melhor aproximação, usa-se a **regra delta** com o algoritmo do gradiente descendente
 - ▶ Também conhecida como Regra Least Mean Square ou Regra de Widrow-Hoff
 - ▶ A cada iteração, diminui o erro
- ▶ Não há *threshold*
 - ▶ Interfere no aprendizado
 - ▶ Função de ativação precisa ser diferenciável



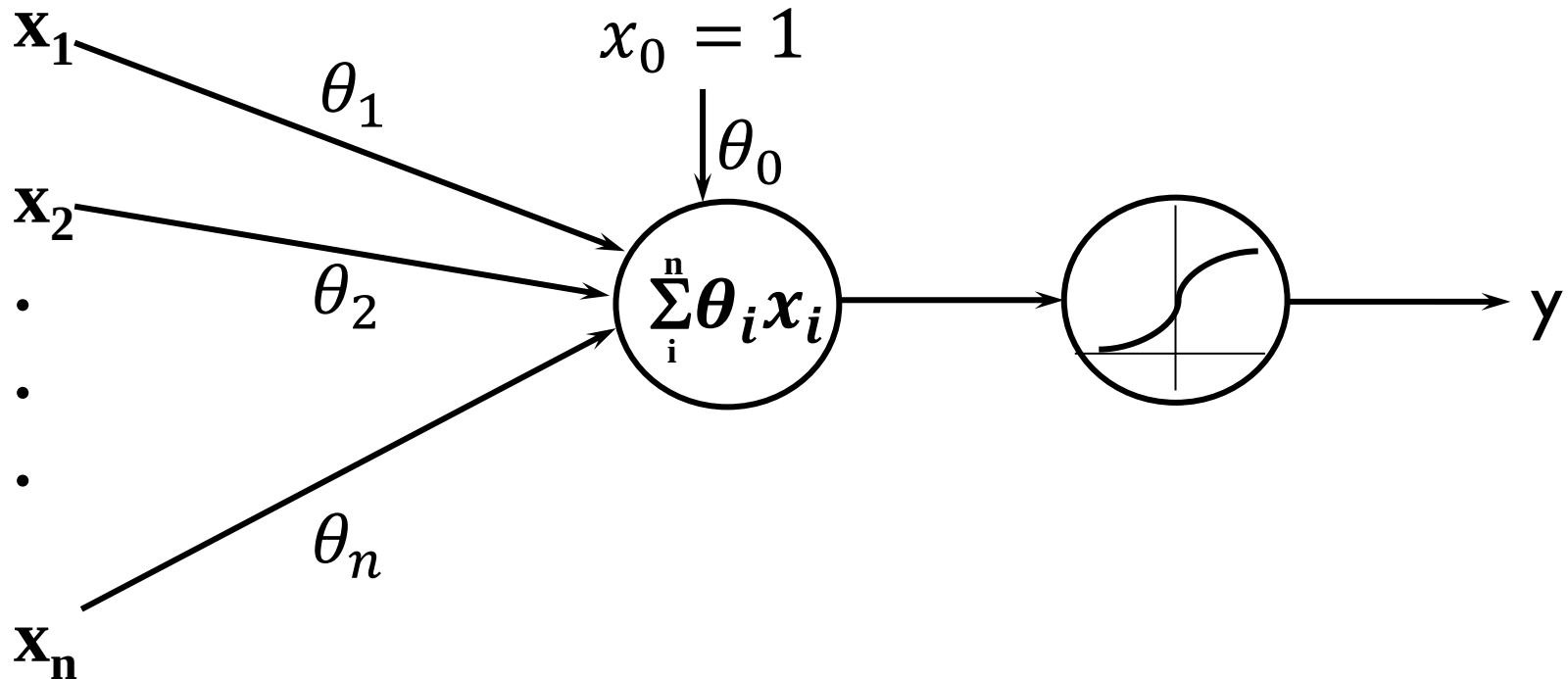
Perceptron – Regra Delta

- ▶ Unidade não é mais conhecida como **perceptron**, mas sim como **unidade linear**
- ▶ Todos os erros de todas as entradas são somados e, somente depois, os pesos são atualizados

$$\theta_j = \theta_j + \alpha * \sum_{i=1}^m (y^{(i)} - h(x^{(i)})) * x_j^{(i)}$$

Já vimos um outro tipo de unidade...

► Unidade sigmóide



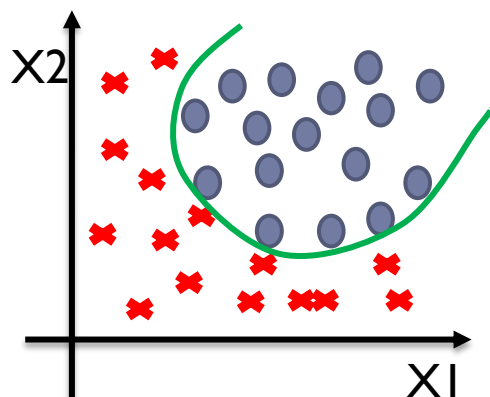
Unidade sigmóide

- ▶ Um regressor logístico é uma unidade sigmóide
- ▶ Geralmente, é o mais utilizado
- ▶ Também tem a limitação de resolver problemas apenas linearmente separáveis. Como resolver?
 1. **Polinômio de maior grau**
 2. **Combinação de regressores logísticos**

Classificação não-linear

Polinômio de maior grau

- ▶ Funciona bem para poucos atributos
- ▶ Exemplo: termos quadráticos (segunda ordem)



$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2$$

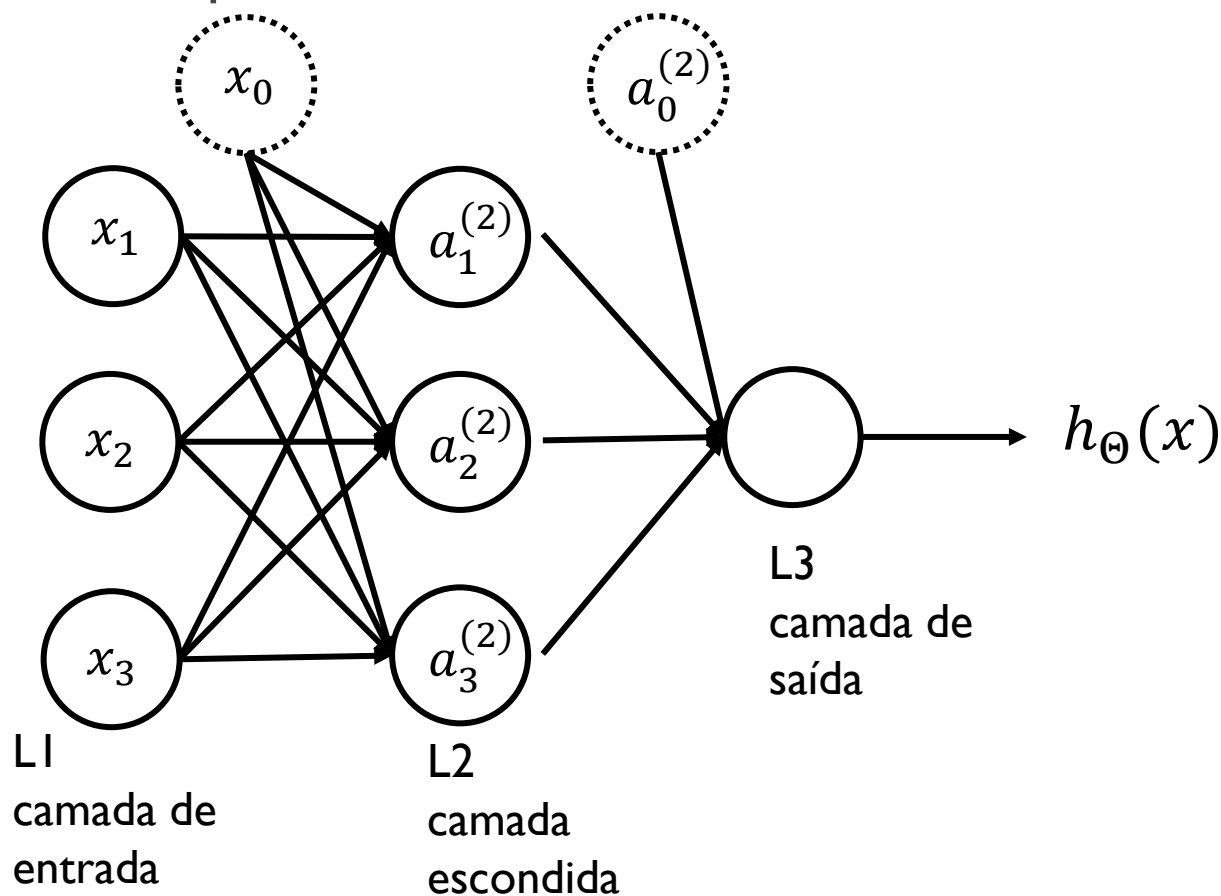
- ▶ Com $n = 100$, tem-se um polinômio de ≈ 5000 termos - $O(n^2)$
- ▶ Com um polinômio de grau 3, tem-se ≈ 170000 termos - $O(n^3)$
- ▶ **Computacionalmente muito custoso para muitos atributos**

Classificação não-linear

Combinação de unidades logísticas

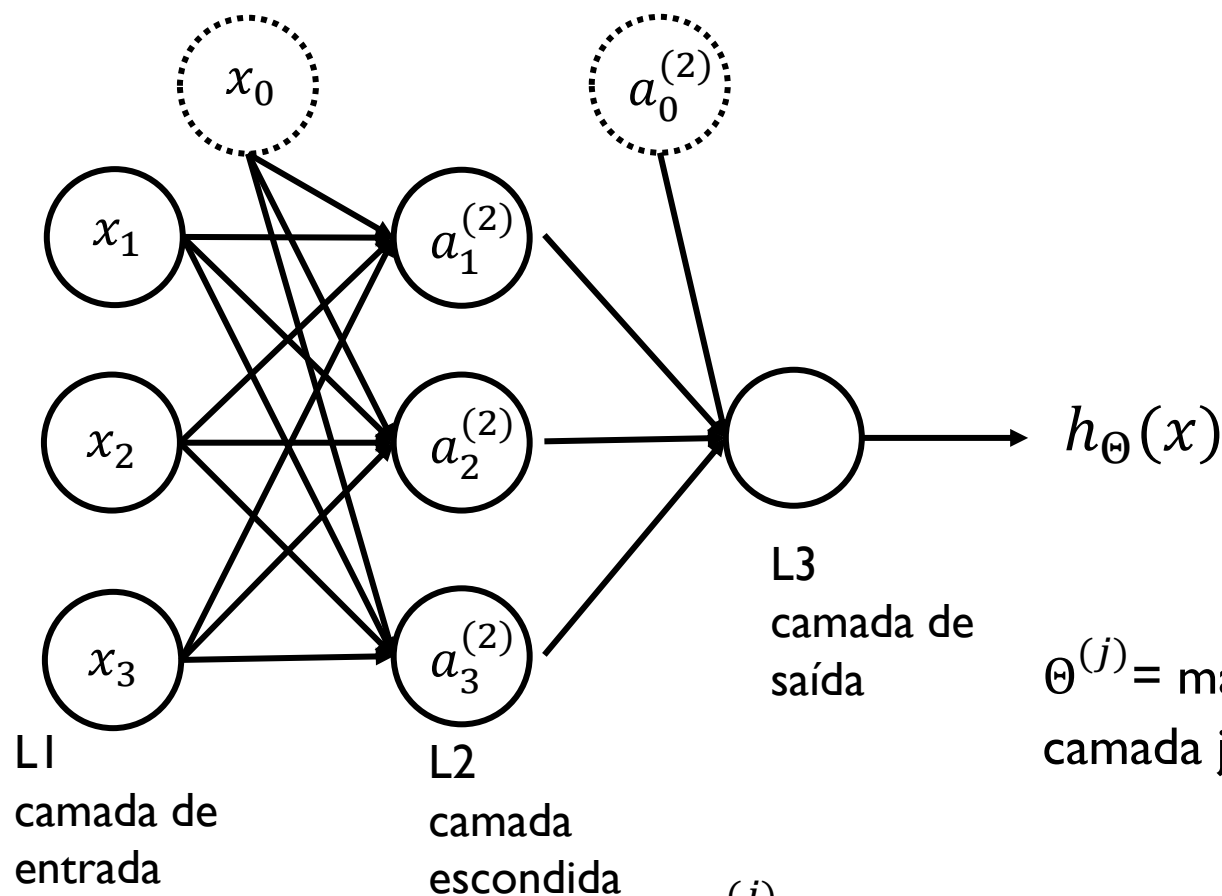
► Redes neurais

- Neurônios dispostos em diversas “camadas”



Classificação não-linear

Combinação de unidades logísticas



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$h(x) = \frac{1}{1 + e^{-\theta^T x}}$$

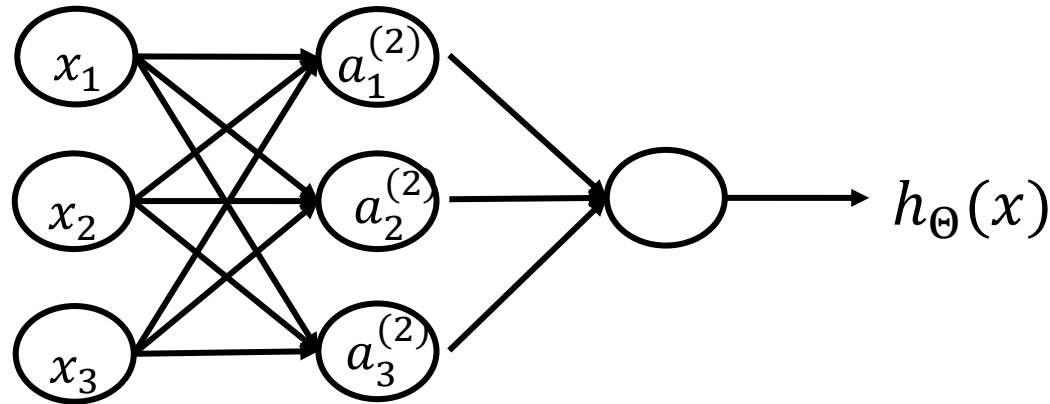
$\Theta^{(j)}$ = matriz de parâmetros ligando a camada j à camada $j+1$

$a_i^{(j)}$ = ativação do nó i na camada j
(saída da função de ativação daquele nó)

Matriz de parâmetros

- ▶ Cada matriz $\Theta^{(j)}$
 - ▶ Número de colunas = número de unidades da próxima camada (s_{j+1})
 - ▶ Número de linhas = número de unidades da camada atual + 1 (devido à unidade de bias) ($s_j + 1$)
 - ▶ $\Theta^{(j)} = [s_j + 1 \times s_{j+1}]$
 - ▶ Se tivermos $s_j = 4$ e $s_{j+1} = 3$, teremos
 - ▶ $\Theta^{(j)} = [5 \times 3]$

Valores de ativação



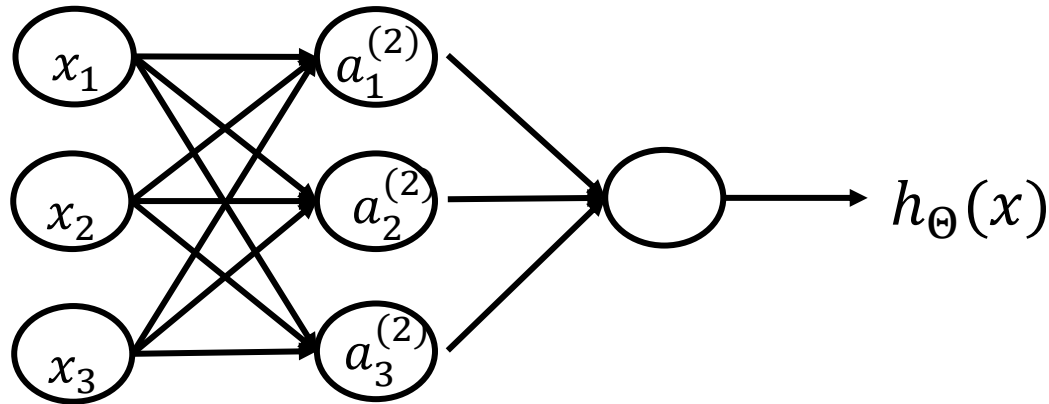
$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

Valores de ativação

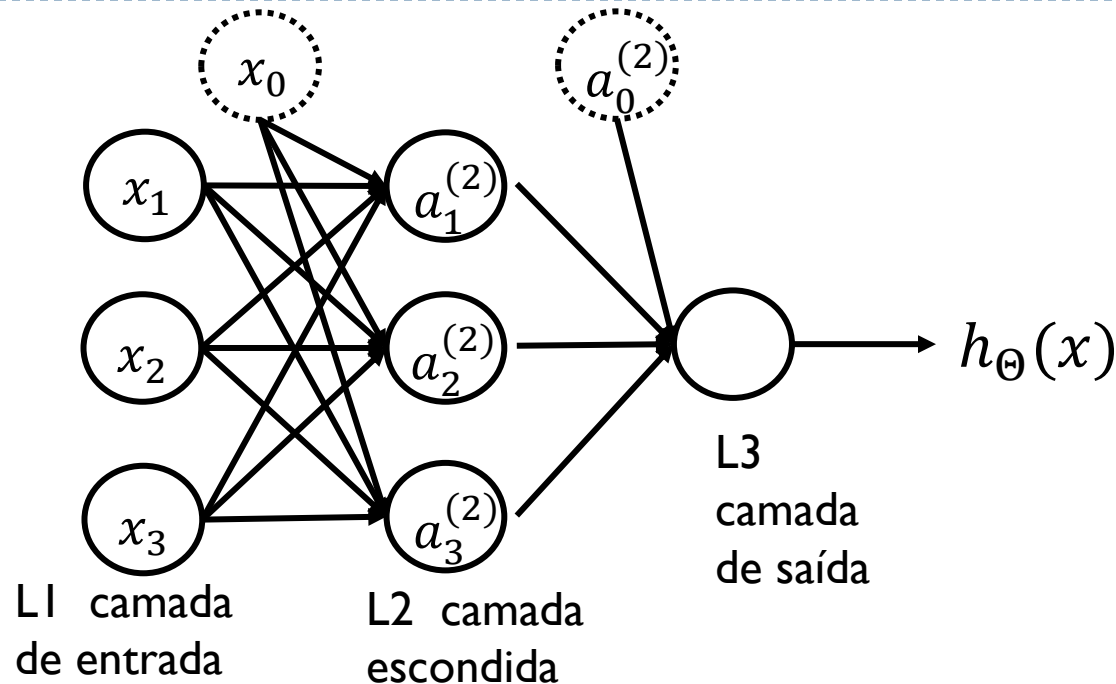


- ▶ Três unidades de entrada, três na camada escondida
 - ▶ $\Theta^{(1)} = [4 \times 3]$
- ▶ Três unidades na camada escondida, uma na saída
 - ▶ $\Theta^{(2)} = [4 \times 1]$
- ▶ Todos os nós de uma camada j são conectados aos nós da camada consecutiva $j+1$

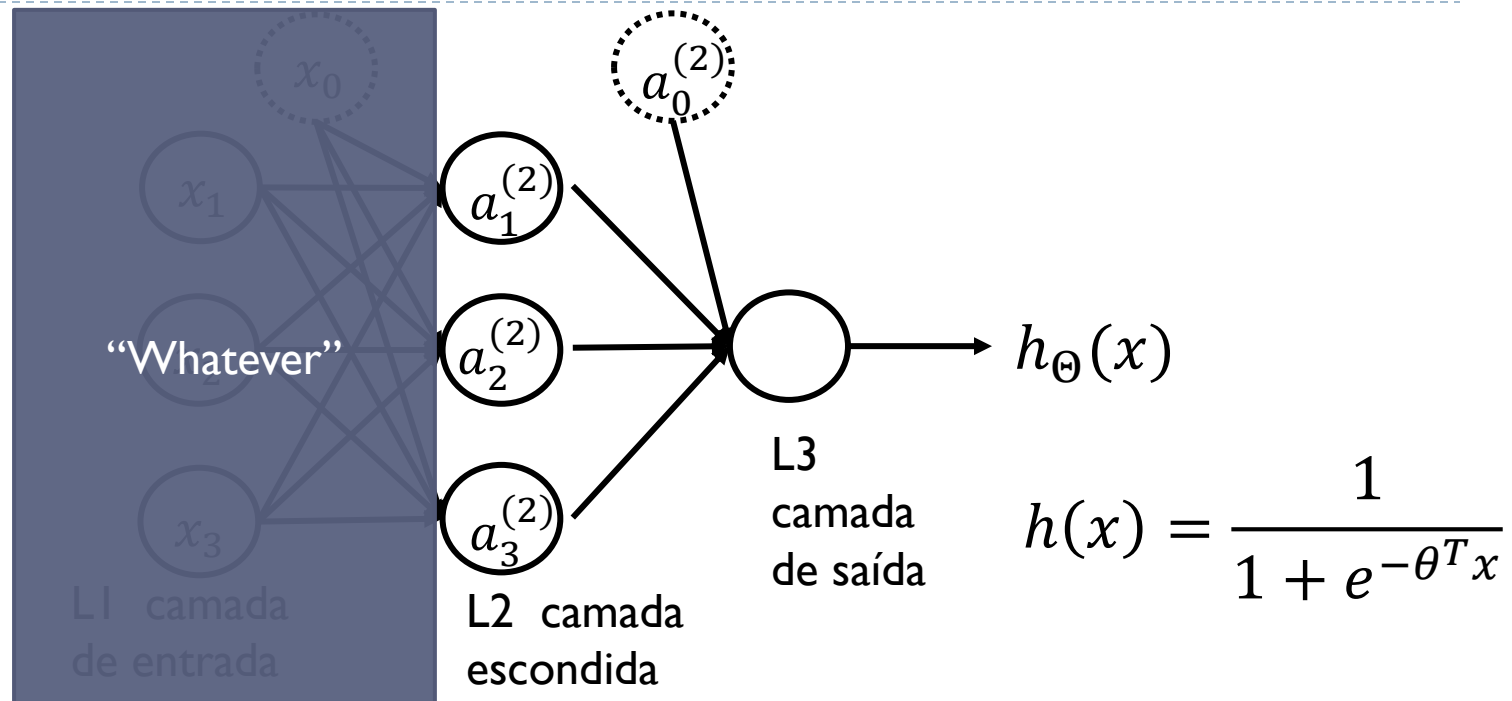
Forward propagation

- ▶ Este processo de propagação dos sinais é conhecido como *forward propagation*
 - ▶ Começa com as ativações das unidades de entrada (valores dos atributos)
 - ▶ Propaga estes sinais e calcula a ativação (saída da função sigmoide) de cada camada sequencialmente
 - ▶ *Implementation trick*: facilita a vetorização

Redes neurais aprendendo seus próprios atributos



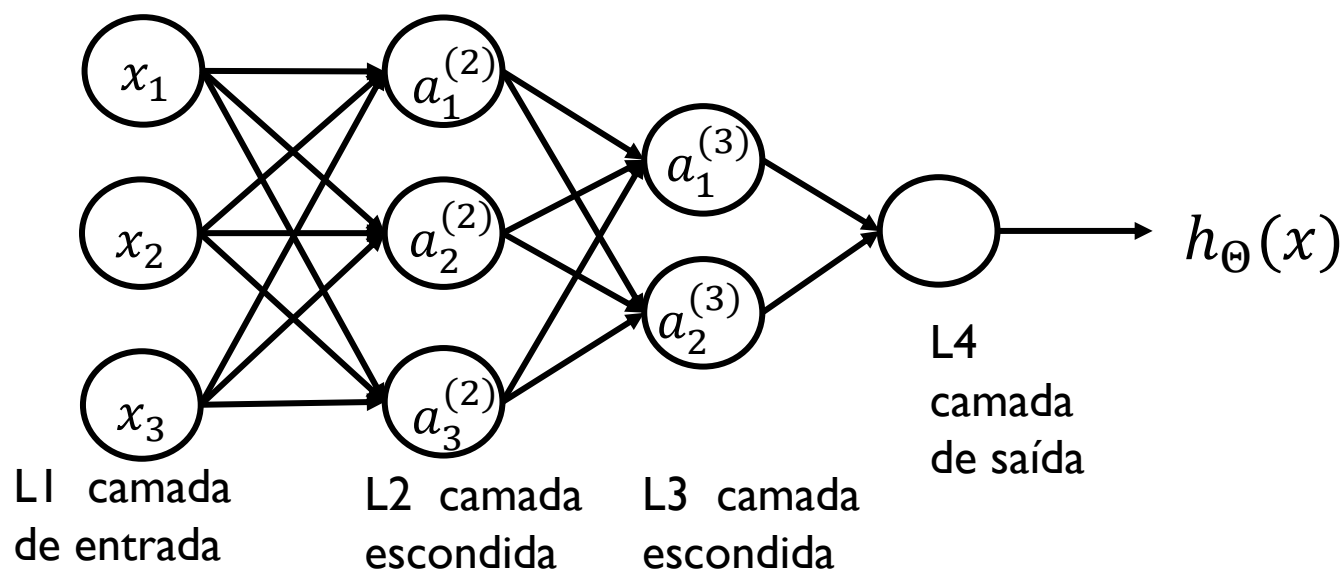
Redes neurais aprendendo seus próprios atributos



- ▶ Regressão logística em função de $a_1^{(2)}$, $a_2^{(2)}$ e $a_3^{(2)}$
 - ▶ Não são os atributos iniciais
 - ▶ São atributos aprendidos na camada anterior
 - ▶ Dependem de $\Theta^{(1)}$
 - ▶ **Classificação não linear!**

Diferentes topologias

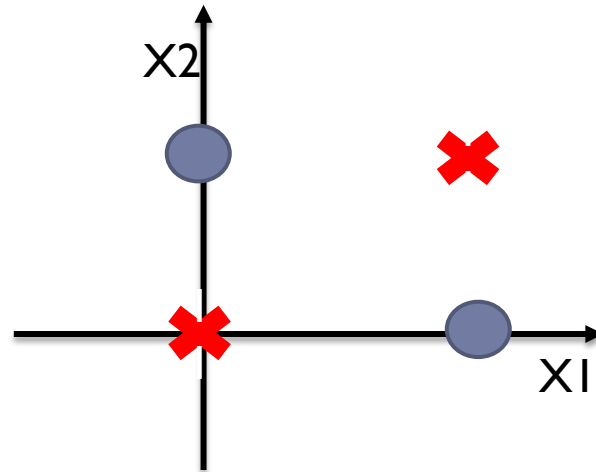
- ▶ É possível distribuir os nós em diferentes topologias
 - ▶ Diferentes números de camadas
 - ▶ Diferentes números de unidades por camada
 - ▶ Quanto maior o número de camadas / unidades, mais complexa é a hipótese aprendida



Exemplo

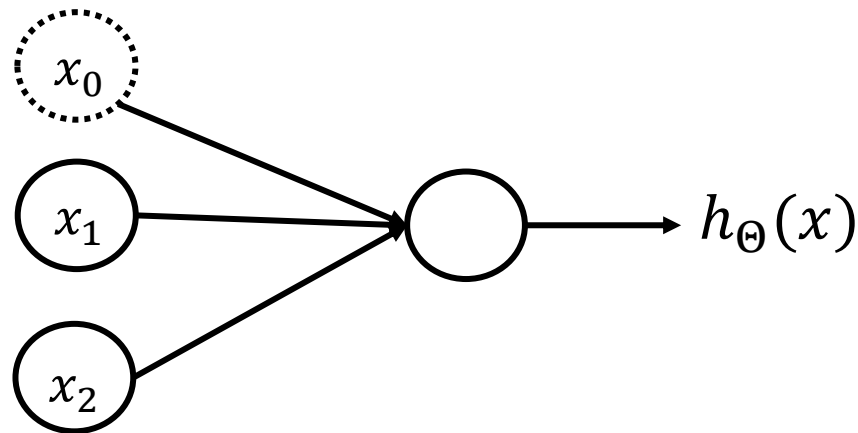
Funções complexas (não lineares)

- ▶ Função XNOR (NOT XOR)
 - ▶ Positivo quando (1,1) ou (0,0)
 - ▶ Borda de decisão não-linear



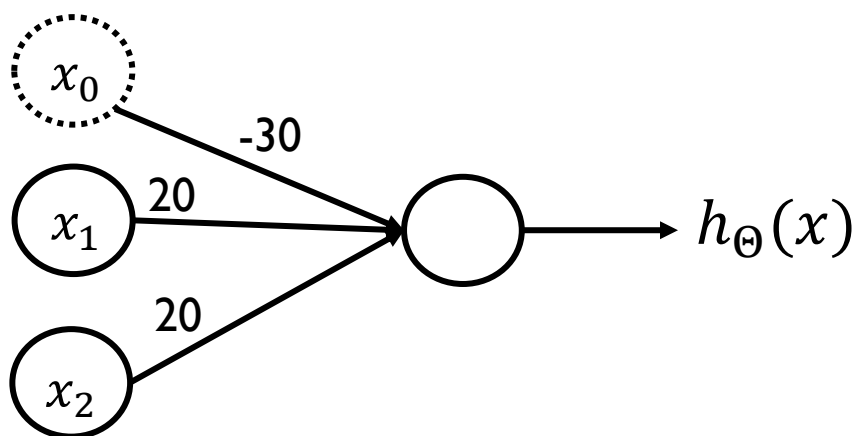
Função AND

- ▶ É possível fazer com que uma rede neural com um único neurônio compute a função AND?

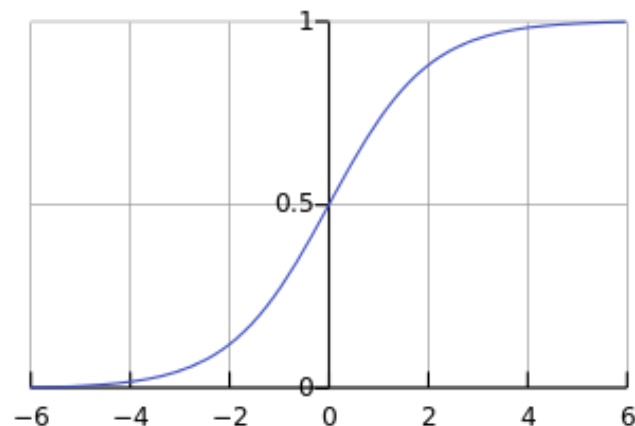


Função AND

- É possível fazer com que uma rede neural com um único neurônio compute a função AND?



$$h_{\Theta}(x) = g(-30 + 20x_1 + 20x_2)$$

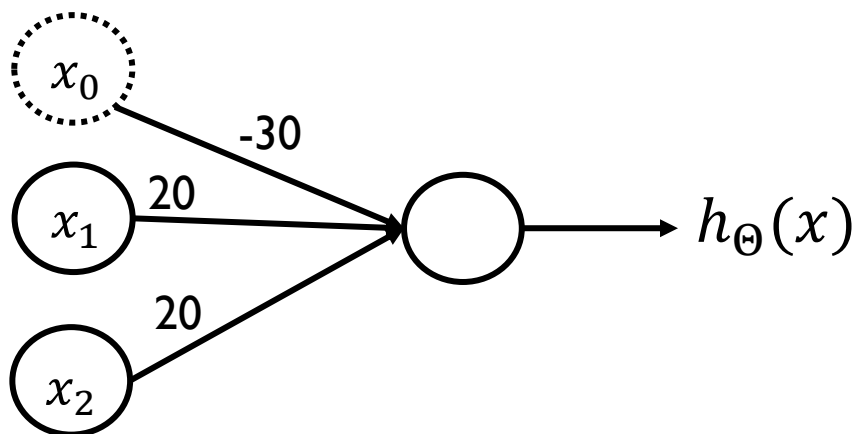


Se $z = 4,6$ $g(z) \approx 0,99$

Se $z = -4,6$ $g(z) \approx 0,01$

Função AND

- É possível fazer com que uma rede neural com um único neurônio compute a função AND?

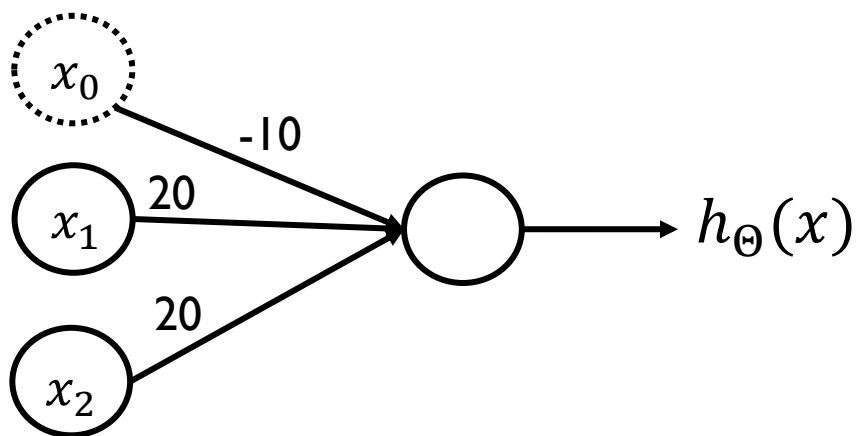


x_1	x_2	$h_{\Theta}(x)$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

$$h_{\Theta}(x) = g(-30 + 20x_1 + 20x_2)$$

$$h_{\Theta}(x) \approx x_1 \text{ and } x_2$$

Função OR



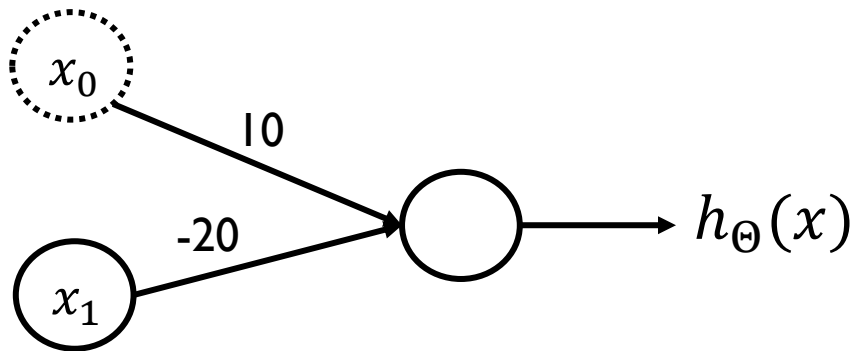
x_1	x_2	$h_{\Theta}(x)$
0	0	$g(-10) \approx 0$
0	1	$g(10) \approx 1$
1	0	$g(10) \approx 1$
1	1	$g(30) \approx 1$

$$h_{\Theta}(x) = g(-10 + 20x_1 + 20x_2)$$

$$h_{\Theta}(x) \approx x_1 \text{ or } x_2$$

Função NOT

► Negação de um valor

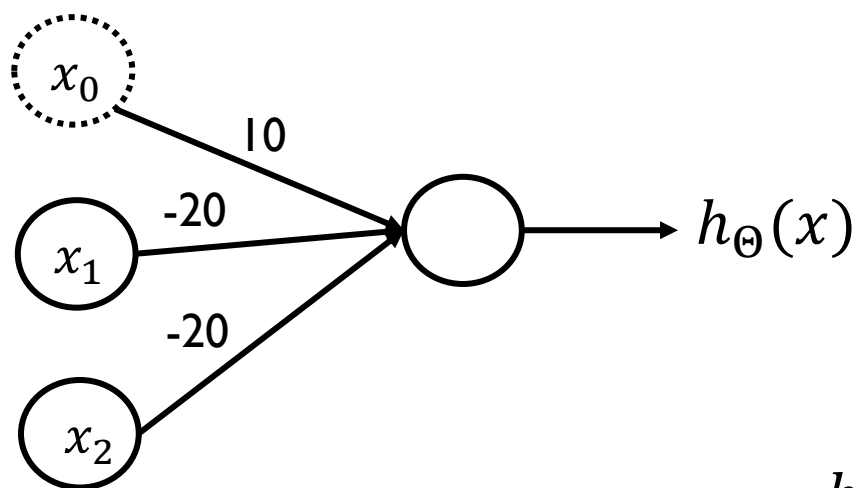


x_1	$h_{\Theta}(x)$
0	$g(10) \approx 1$
1	$g(-10) \approx 0$

$$h_{\Theta}(x) \approx !x_1$$

Função (NOT x_1) AND (NOT x_2)

- Vai ser igual a 1 somente quando $x_1 = x_2 = 0$

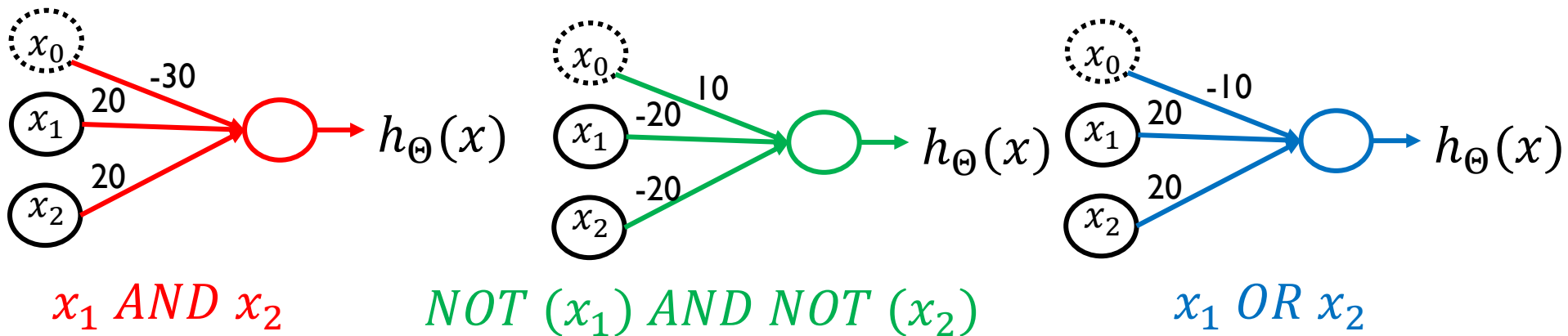


x_1	x_2	$h_{\Theta}(x)$
0	0	$g(10) \approx 1$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(-30) \approx 0$

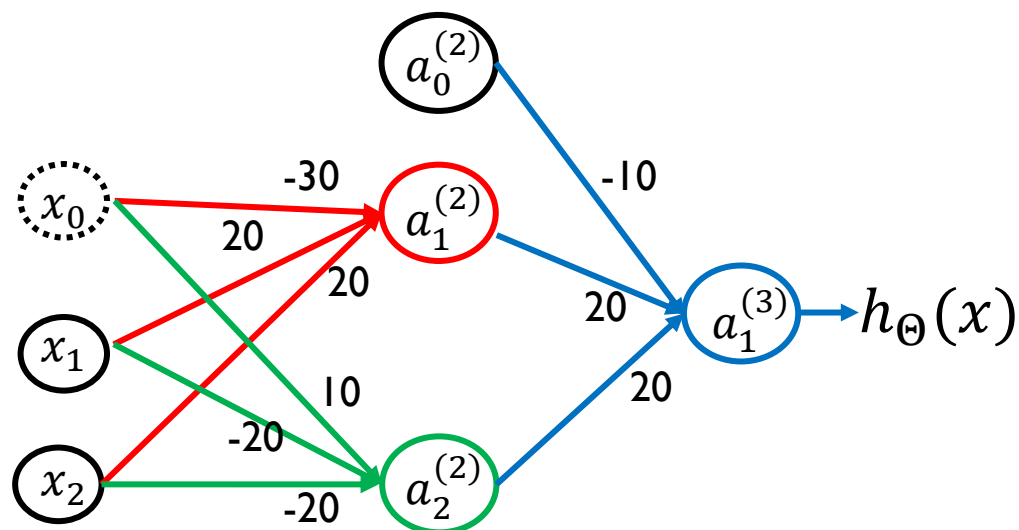
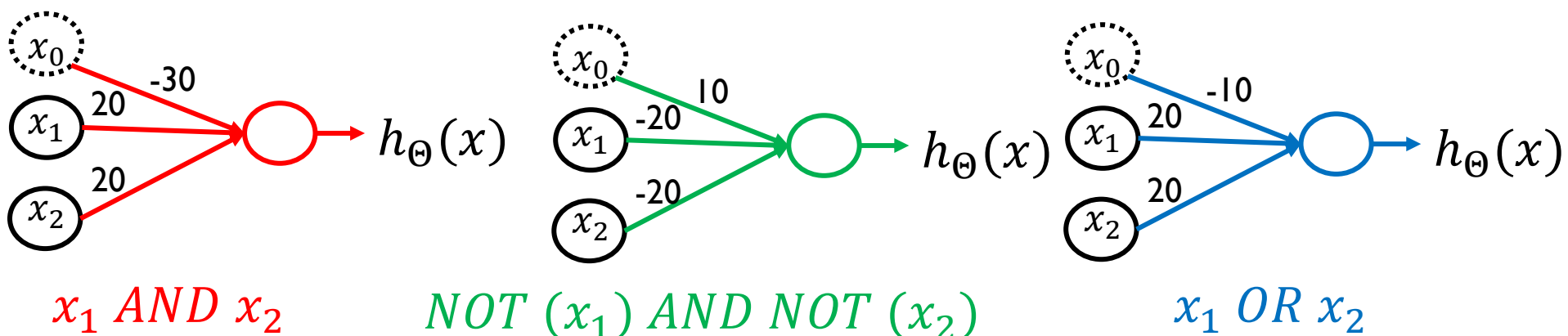
$$h_{\Theta}(x) \approx \text{NOT}(x_1) \text{ AND } \text{NOT}(x_2)$$

$$h_{\Theta}(x) = g(10 - 20x_1 - 20x_2)$$

Juntando tudo: função XNOR



Juntando tudo: função XNOR



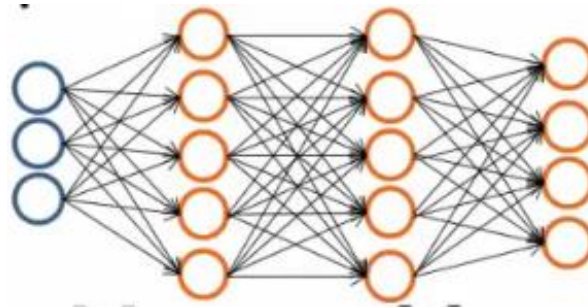
x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{\Theta}(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

Classificação multiclasse

- ▶ Redes como vimos resolvem problemas binários
 - ▶ Dois valores de classe
 - ▶ Um codificado como 0, outro codificado como 1
- ▶ Entretanto, é comum encontrarmos problemas multiclasse
 - ▶ Vários valores de classe
 - ▶ Exemplo: distinguir dígitos
 - ▶ 10 possíveis classes (0 – 9)
 - ▶ Impossível classificar com um único neurônio na camada de saída

Classificação multiclasse

- ▶ **Solução: utilizar abordagem um contra todos (*one vs. all*)**
 - ▶ Número de neurônios na camada de saída é igual ao número de classes
 - ▶ Saída passa a ser codificada não por um valor binário, mas sim por um vetor de valores binários
 - ▶ Exemplo: uma rede para reconhecimento de pedestres, carros, motos ou caminhões
 - ▶ Vetor de 4 posições
 - $[1,0,0,0]$ = carro
 - $[0,1,0,0]$ = moto
 - $[0,0,1,0]$ = caminhão
 - $[0,0,0,1]$ = pedestre

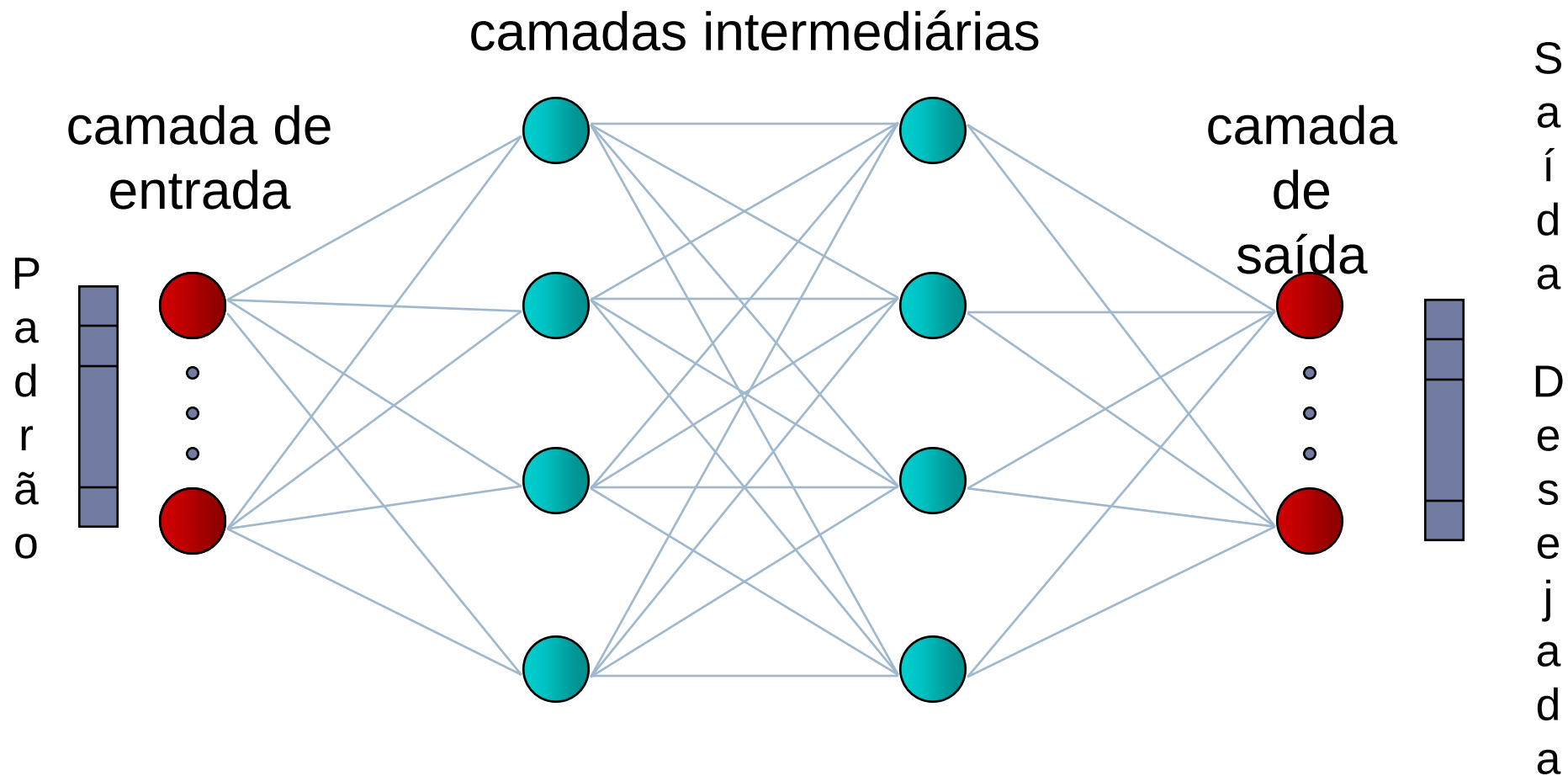


$$h_{\Theta}(x) \in \mathbb{R}^4$$

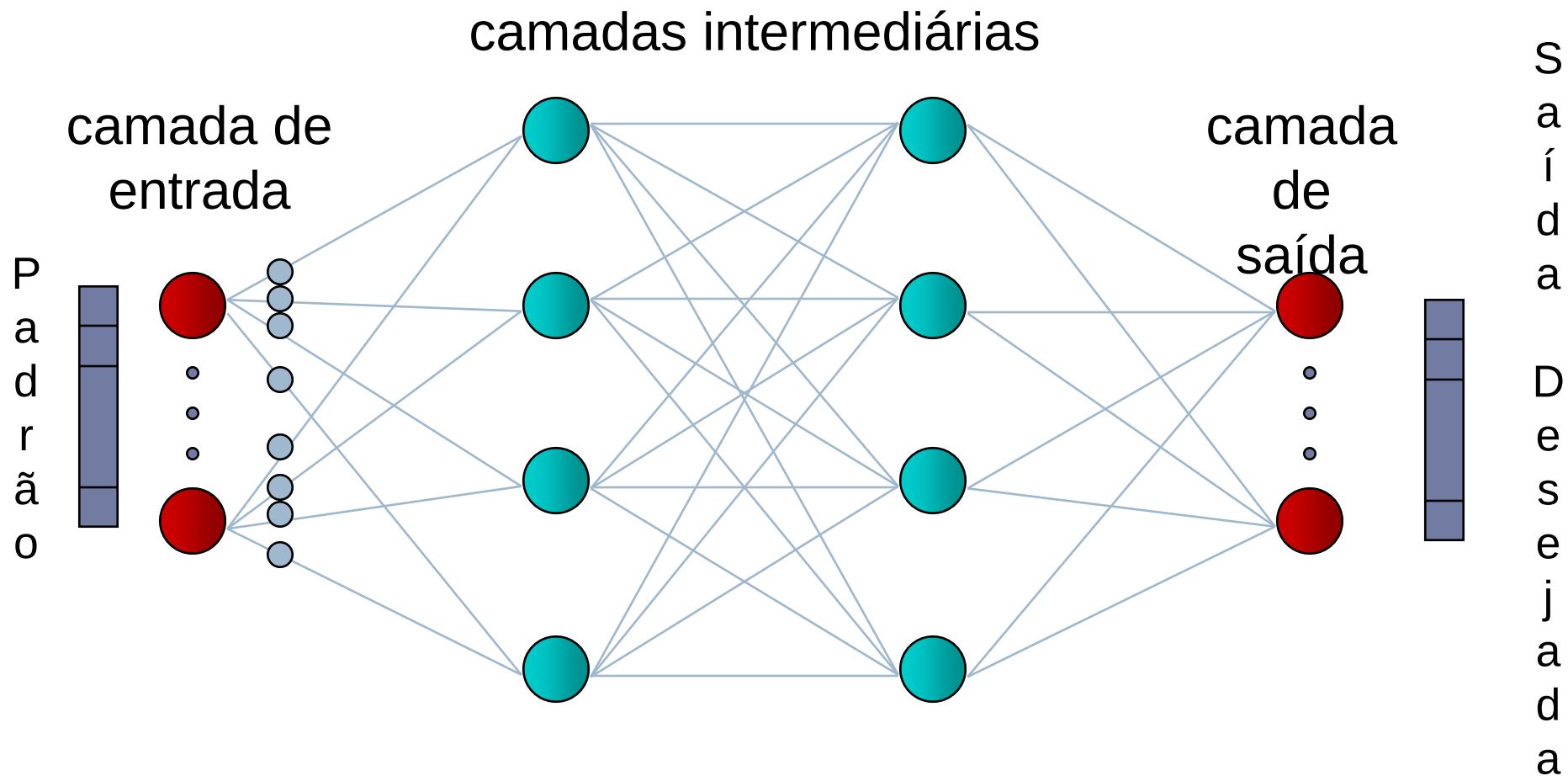
Retropropagação dos erros

- ▶ Algoritmo Backpropagation é utilizado para aprendizado das redes neurais
- ▶ Visa à minimização da função de custo
- ▶ Duas fases: *forward* e *backward*
 - ▶ Forward
 - ▶ Propagação dos sinais de ativação das unidades
 - ▶ Da camada de entrada para a camada de saída
 - ▶ Já vimos antes...
 - ▶ Backward
 - ▶ Propagação dos erros para ajustes dos pesos
 - ▶ Da camada de saída para a camada de entrada

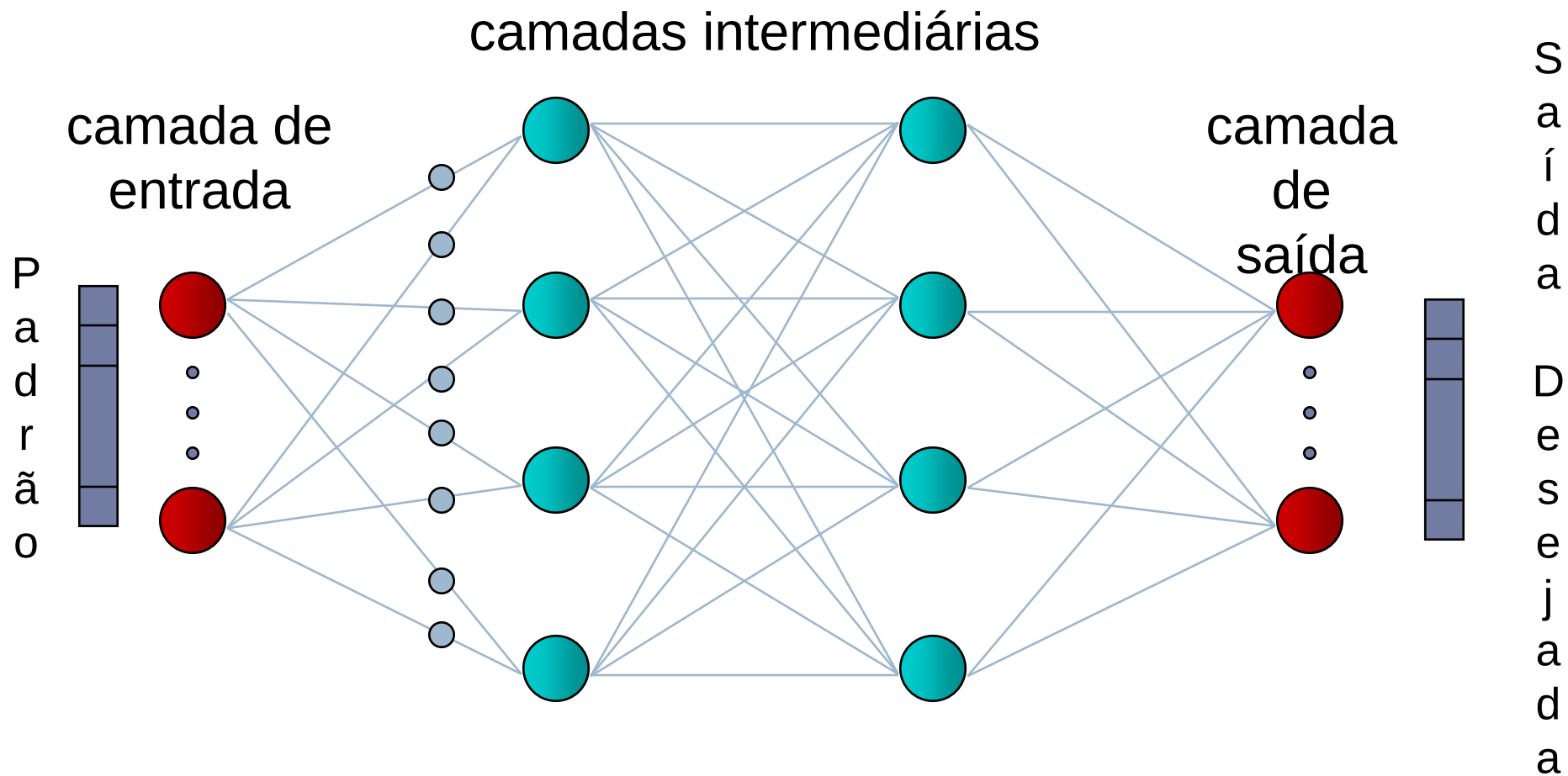
Backpropagation - Intuição



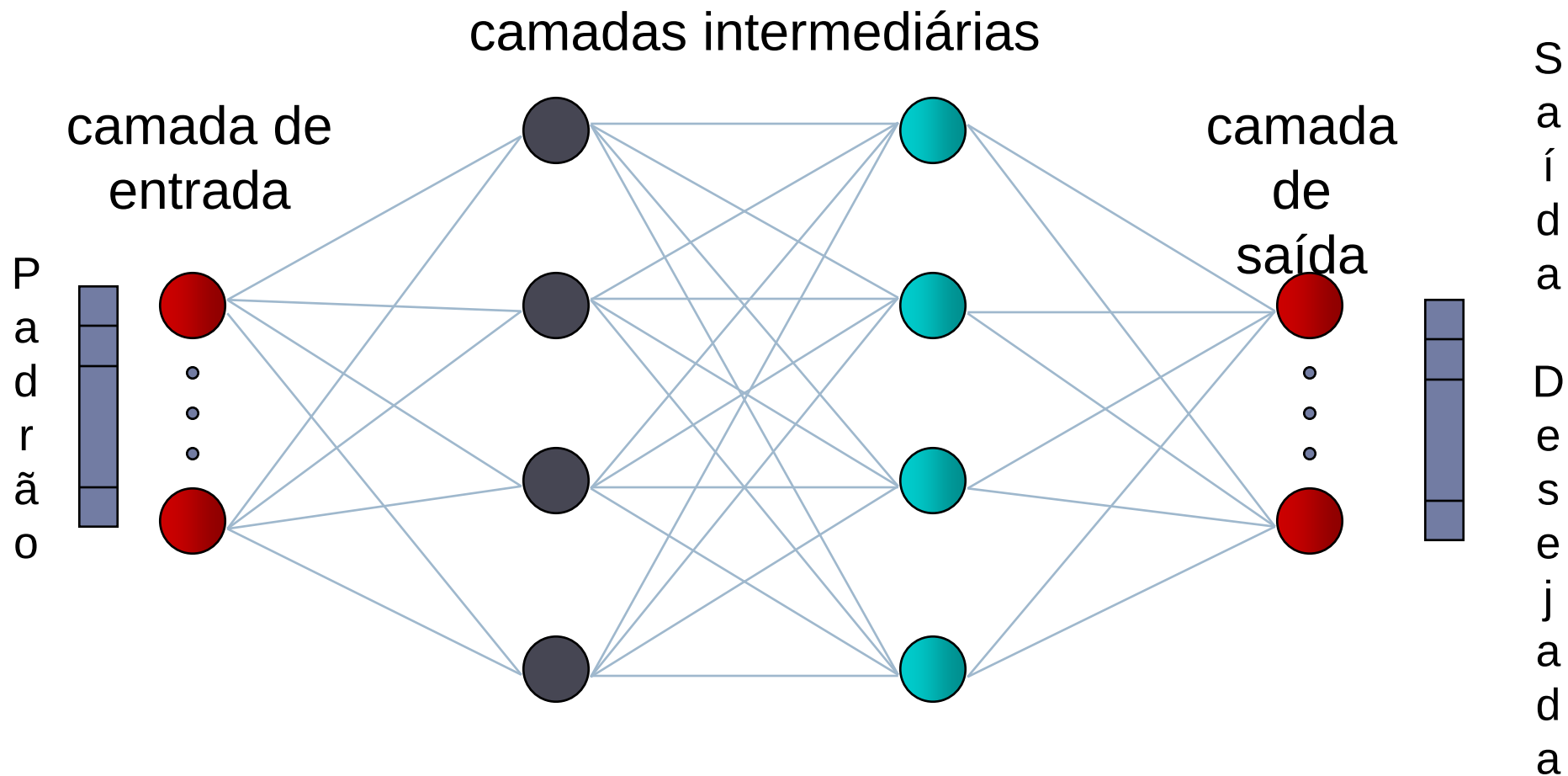
Backpropagation - Intuição



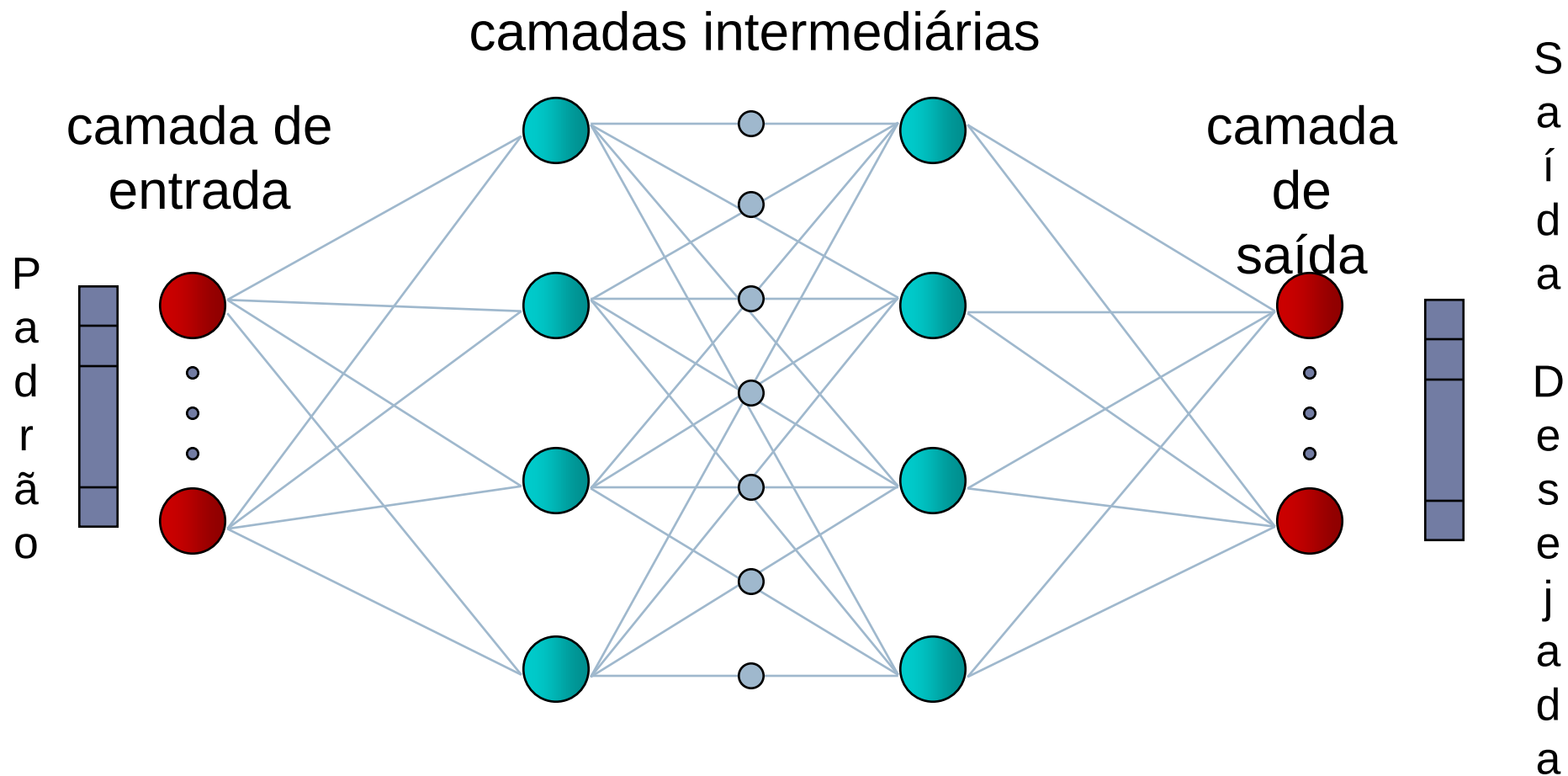
Backpropagation - Intuição



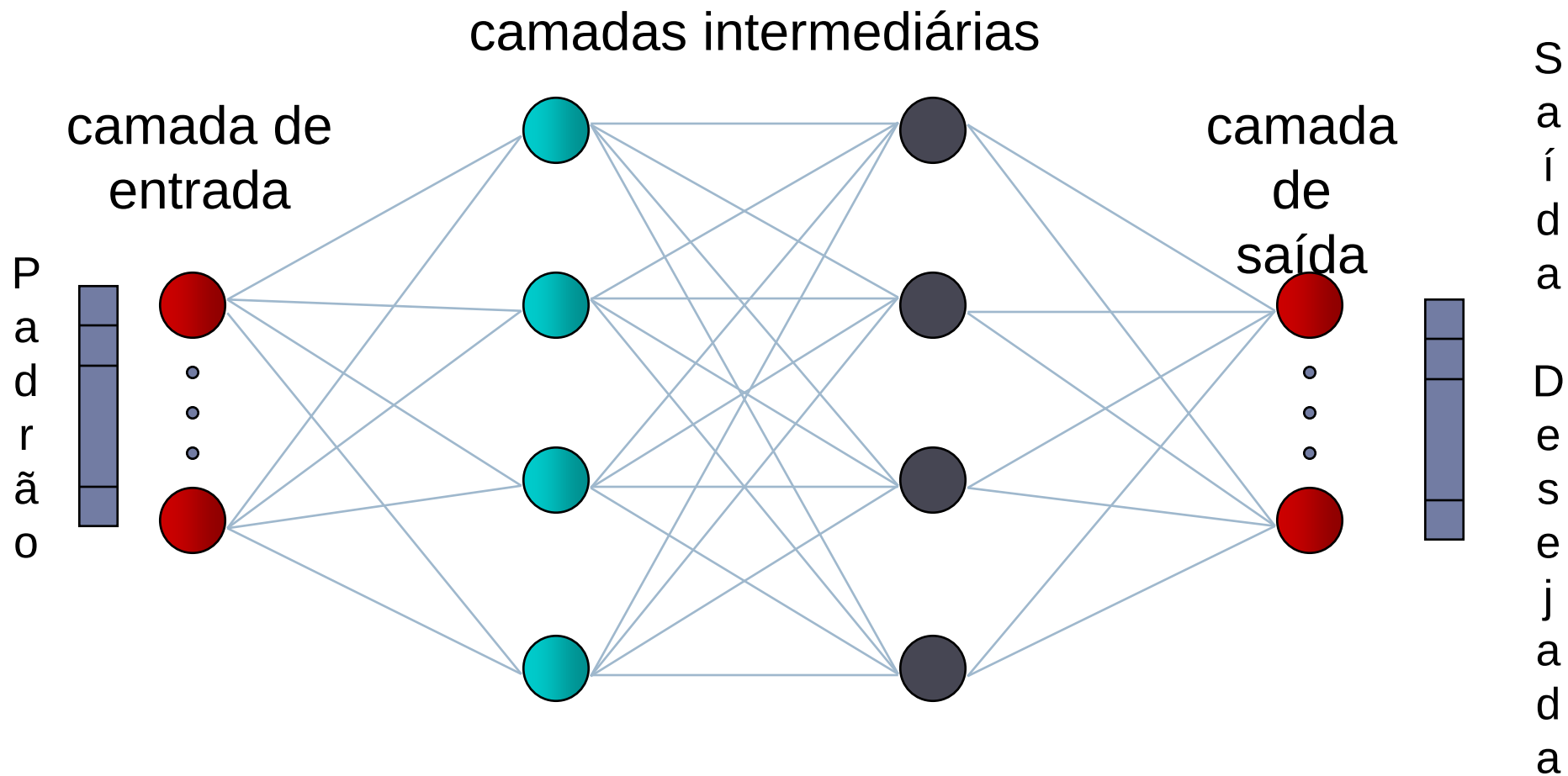
Backpropagation - Intuição



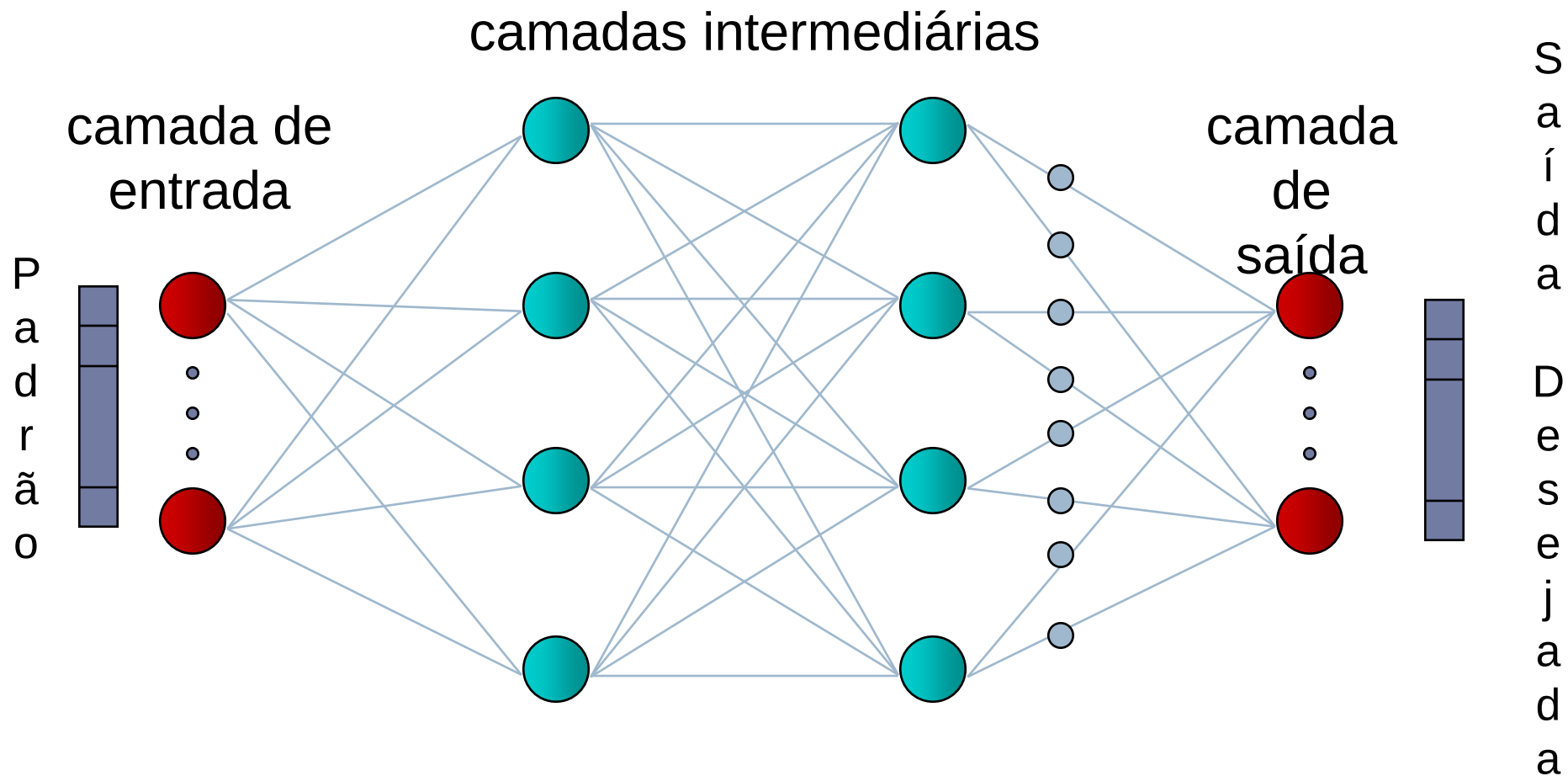
Backpropagation - Intuição



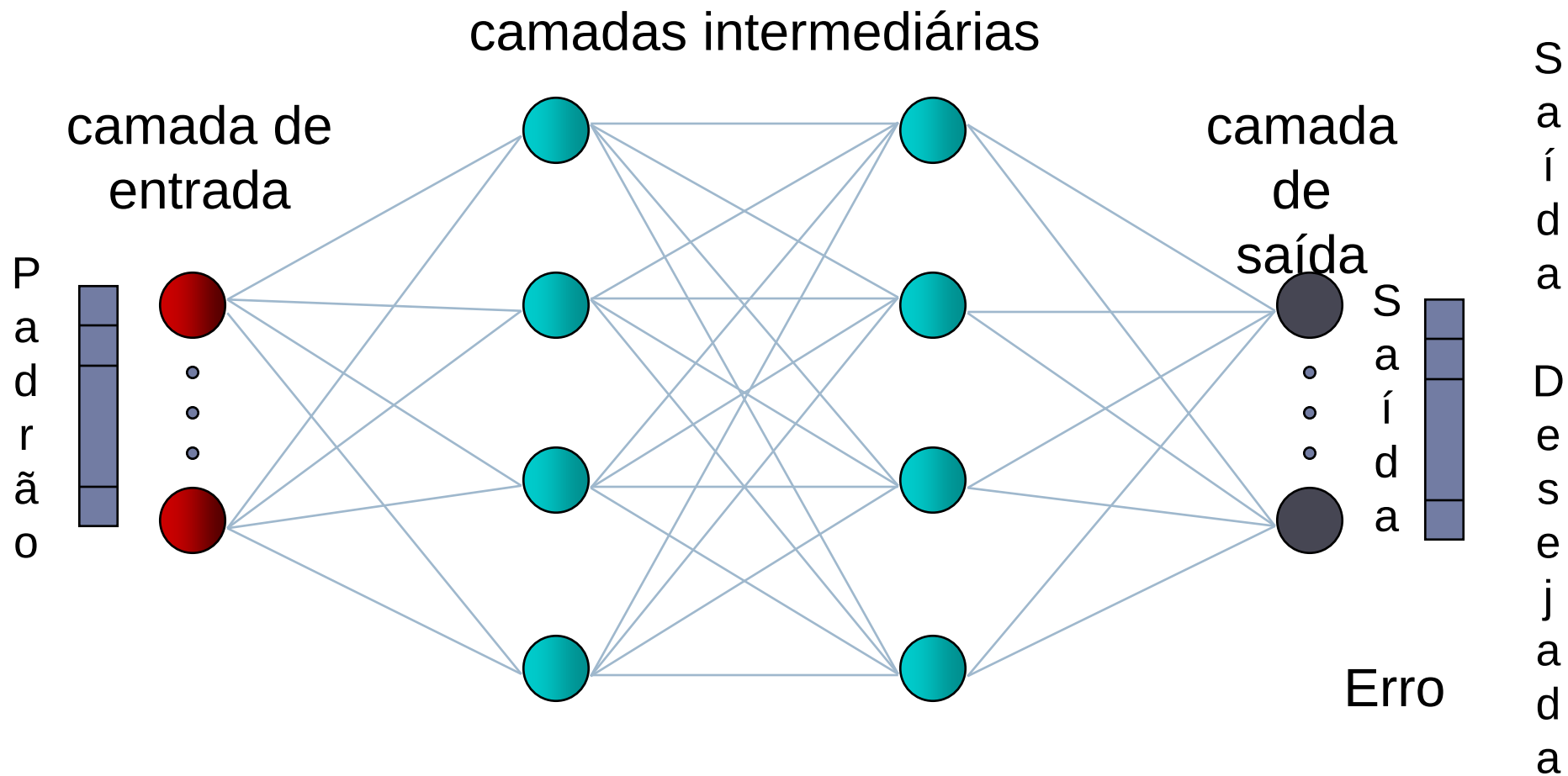
Backpropagation - Intuição



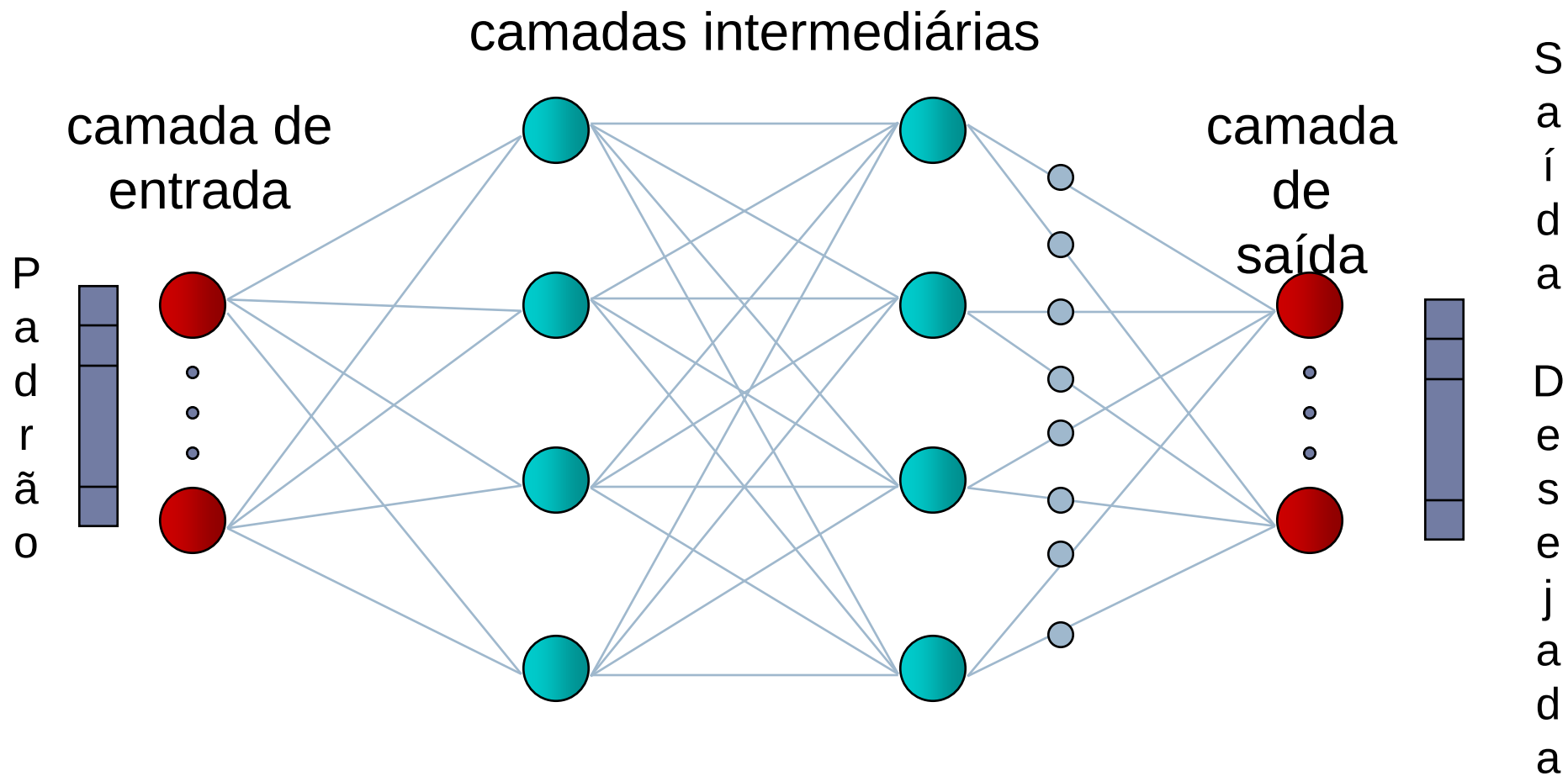
Backpropagation - Intuição



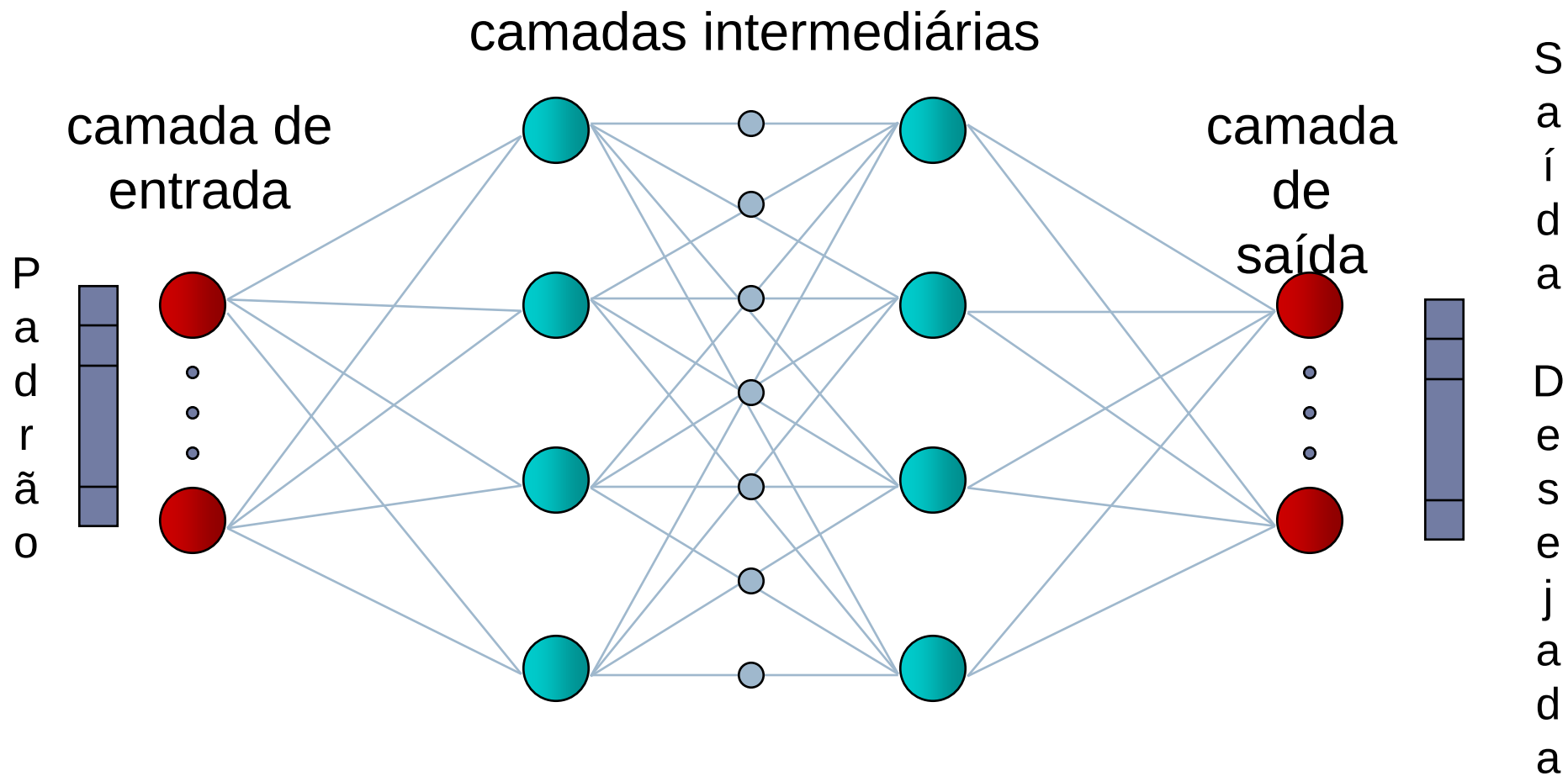
Backpropagation - Intuição



Backpropagation - Intuição



Backpropagation - Intuição



Backpropagation - Intuição

